

Software Design Specification

Guthrie Document Centre (CRS)

SD Guthrie · SharePoint Online document management system

Repository sd-gathrie · SPFx 1.23 · package version 1.0.580.0

Live site <https://sdguthrie.sharepoint.com/sites/CRS>

Data structures captured from the live site on 2026-10-01

Document version 1.0 · 1 October 2026 · Prepared by Trinergy Digital

Document control

Version	Date	Author	Change
1.0	1 October 2026	Trinergy Digital	First issue, for handover to the development team.

This specification replaces the earlier CRS Developer Guide. The data structures in chapter 4 and the security data in chapter 5 were read from the live site, not from the code, so they describe what is really deployed.

Contents

Document control.....	2
Contents	3
1. Introduction	6
1.1 Purpose	6
1.2 Scope	6
1.3 How to read this document.....	6
1.4 Definitions and abbreviations.....	6
1.5 References	7
2. System overview.....	8
2.1 What the system does	8
2.2 Users	8
2.3 System context	9
2.4 Design principles.....	10
3. Architecture	11
3.1 Technology stack	11
3.2 Logical architecture	11
3.3 Deployment architecture.....	13
3.4 Solution components.....	14
3.5 Pages.....	15
3.6 Code organisation.....	15
3.7 Runtime behaviour common to all web parts	16
4. Data design	17
4.1 Where data is stored	17
4.2 Logical data model.....	17
4.3 Libraries	19
4.4 Document columns.....	22
4.5 CRS lists.....	26
4.6 Configuration data	29
4.7 Term store	31
4.8 Folder model.....	32
4.9 Record lifecycles	33
4.10 Audit events.....	34
4.11 Volumes and limits	35
5. Security design.....	36
5.1 Identity and authentication	36

5.2 Authorisation model.....	36
5.3 Personas and roles.....	38
5.4 Roles, permission levels and libraries.....	39
5.5 Permission levels	39
5.6 Where permissions are applied (live)	40
5.7 Service account and proxy actions	41
5.8 Highly Confidential separation	41
5.9 Security rules for developers.....	41
6. Component design.....	43
6.1 Upload Form	43
6.2 Bulk Upload.....	44
6.3 Approval Document and the Approve or reject panel	44
6.4 My Submissions and Document-Viewer	45
6.5 Requests	45
6.6 Search	46
6.7 Folder Administration and Folder Reconciliation	46
6.8 Folder Structure and Migration	47
6.9 Group Management	48
6.10 Settings, Configuration and Access pages	48
6.11 Audit Log.....	49
6.12 Extensions.....	49
7. Process design	51
7.1 Upload and tagging.....	51
7.2 Approval and routing.....	53
7.3 Bulk import	54
7.4 Approver notifications and reminders.....	54
7.5 Delete request	56
7.6 Share request and expiry	58
7.7 Seven-year archive.....	59
7.8 Folder reconciliation	60
7.9 Adding a unit (end to end).....	61
8. Integration design: Power Automate	62
8.1 Environment and connections.....	62
8.2 Flow inventory	62
8.3 Which flows listen to which list	63
8.4 Data contract between code and flows.....	63
8.5 Emails.....	64

8.6 List GUIDs the flows use (CRS)	65
9. Interface design	66
9.1 User interface map	66
9.2 SharePoint APIs used	66
9.3 REST conventions.....	67
10. Non-functional design	68
11. Design decisions.....	69
12. Known issues and open items	70
12.1 Ready to roll out on CRS (tested on the test site)	70
12.2 Waiting on someone.....	70
12.3 Known gaps (accepted for now)	70
Appendix A. Build, run and deploy	71
A.1 Software and access	71
A.2 Run locally	71
A.3 Build.....	72
A.4 Deploy.....	72
A.5 Register the extensions	73
Appendix B. Set up a test site	74
Appendix C. Troubleshooting	75
Appendix D. Flow operations.....	76
D.1 Rules for editing flows.....	76
D.2 If gdc's password changes	76
D.3 Export and back up.....	76
D.4 Import or update.....	76
D.5 Point a flow at another site.....	76
D.6 Make the HC copy of a flow	76
D.7 Rebuild a generated flow	77
D.8 Turn a flow off safely.....	77

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes how the Guthrie Document Centre (CRS) is built: its architecture, where every piece of data is stored and how it is structured, the security model, each component, the processes that move a document through its life, the Power Automate integration, and the design decisions behind them. It is written so that a development team can maintain, extend, rebuild or audit CRS without the original developer.

1.2 Scope

- **In scope:** the SharePoint Framework (SPFx) solution `sd-gathrie` (17 web parts, 2 active extensions), the SharePoint site `/sites/CRS` it runs on (libraries, lists, term store, groups, pages), and the 25 Power Automate flows that run as the service account.
- **Out of scope:** Microsoft 365 tenant administration, Entra ID, Exchange mail-flow rules and the SD Guthrie network. These are referenced only where CRS depends on them.
- **Sites:** the live site is <https://sdguthrie.sharepoint.com/sites/CRS>. Development and testing use a separate site with the same structure (Appendix B). The two sites have different list GUIDs, term GUIDs and data.

1.3 How to read this document

If you need to...	Read
Understand the system in one sitting	Chapters 2 and 3
Know where data is stored and what each column means	Chapter 4 (data design)
Change who can see or do what	Chapter 5 (security), then 6.9 Group Management
Change a screen	Chapter 6 (that component), then chapter 9 (interfaces)
Change or debug an email, routing or deletion	Chapters 7 and 8
Build, deploy or set up a test site	Appendices A and B
Fix something broken	Appendix C, then chapter 12

1.4 Definitions and abbreviations

Table 1. Definitions

Term	Meaning
CRS	Central Repository System, the product name. Formerly DMS (Document Management System); some code and list names still say DMS.
GDC	Guthrie Document Centre: the site title and the display name of the service account <code>gdc@sdguthrie.com</code> .
SPFx	SharePoint Framework: client-side web parts and extensions, written in TypeScript and React.
Segment	A business segment, the top folder of the tree (Group Head Office = GHO, Minamas Head Office = MHO, NBPOL Head Office = NBPOLHO).
Tier / level	A folder level below the segment. Two are permissioned (normally Department, Unit); the rest (Sub Unit, Year, Document Type) inherit.
Unit	The second permissioned tier and the smallest confidentiality boundary.
Staging / approval library	Library holding uploads waiting for approval (ApprovalDocument, HCAApprovalDocument).

Term	Meaning
Documents / approved library	Library holding approved documents (Shared Documents, HCDocuments).
HC	Highly Confidential. HC documents live in their own library pair.
PIC	Person in charge: an uploader.
HoU, HoD	Head of Unit (approver), Head of Department.
Persona	A named bundle of roles given to a SharePoint group (e.g. Head of Unit = APR + SHARE + UPL).
Role	A capability code stored in CRS Group Map (UPL, APR, SHARE, MEMBER, DEPTVIEW, SEGVIEW, GLOBAL and HC variants).
Term / term GUID	A managed-metadata term in the site term store. Its GUID is the stable key used across CRS.
Abbreviation	Short code used as a folder name for a term (e.g. Group IT → GIT).
Reconciliation	The admin process that builds folders and grants permissions from the configuration lists.
Proxy action	An action a user cannot perform with their own permissions, done for them by a flow running as gdc.
TagPayload	JSON of metadata the upload form leaves on a CRS Submissions row for the tagging flow to apply.
Moderation status	SharePoint content approval state: 0 Approved, 1 Rejected, 2 Pending (field OData__ModerationStatus).

1.5 References

Item	Where
Source code	Git repository sd-gathrie (branch main)
Power Automate exports (source of truth for flows)	PowerAutomateFlowsSDG/*.zip in the repository, exported 25 September 2026
Live schema capture used in chapters 4 and 5	scripts/dump-site-schema.js (read-only browser-console script); captured on 2026-10-01
Flow generators	scripts/build-bundled-approver-flow.py, scripts/build-bundled-reminder-flow.py
Diagnostic scripts	scripts/check-*.js, scripts/diagnose-*.js (read-only); repair-*, purge-* (write, off by default)

2. System overview

2.1 What the system does

CRS is a document management system on SharePoint Online. Staff file documents into a structured folder tree (segment › department › unit › year › document type). Every upload waits in an approval library until the unit's Head of Unit approves it; Power Automate then moves it into the permanent library. Seven years after creation a document moves to a read-only archive. Every folder carries its own permissions, so a person only sees the units they belong to.

- Upload with metadata (single upload form, and bulk upload for historical documents).
- Approval by the unit's approver, one at a time or in bulk, with email notification and reminders.
- Automatic filing into the approved library, keeping version history when a file is replaced.
- Search across approved and archived documents, security-trimmed by SharePoint.
- Delete and share requests with approval, carried out by the service account.
- A full audit log of uploads, decisions, deletions, shares and admin changes.
- Admin tools: folder structure, folder codes, groups and members, reconciliation, file types, page access.

2.2 Users

Table 2. Personas

Persona	What they do in CRS
PIC (uploader)	Uploads into their unit; reads their unit's approved documents; tracks uploads on My Submissions; raises delete and share requests.
PIC (HC)	As PIC, and may file Highly Confidential documents.
Head of Unit (approver)	Approves or rejects uploads for their unit; decides delete and share requests; may upload (own uploads can self-approve).
Head of Unit (HC)	As Head of Unit, including the HC libraries. The only persona that approves HC documents.
Head of Department	Reads the whole department; decides share requests in the department.
SDG Employee (viewer)	Reads their unit's approved documents (the HC variant reads HC too).
C-Level (segment / global)	Reads a whole segment, or every segment, read-only, including the archives.
System administrator	Member of the site Owners group (Guthrie Document Centre Owners). Uses the admin pages.
Service account gdc@sdguthrie.com	Owner of every flow; performs proxy actions and sends every email. Must stay in Owners.

2.3 System context

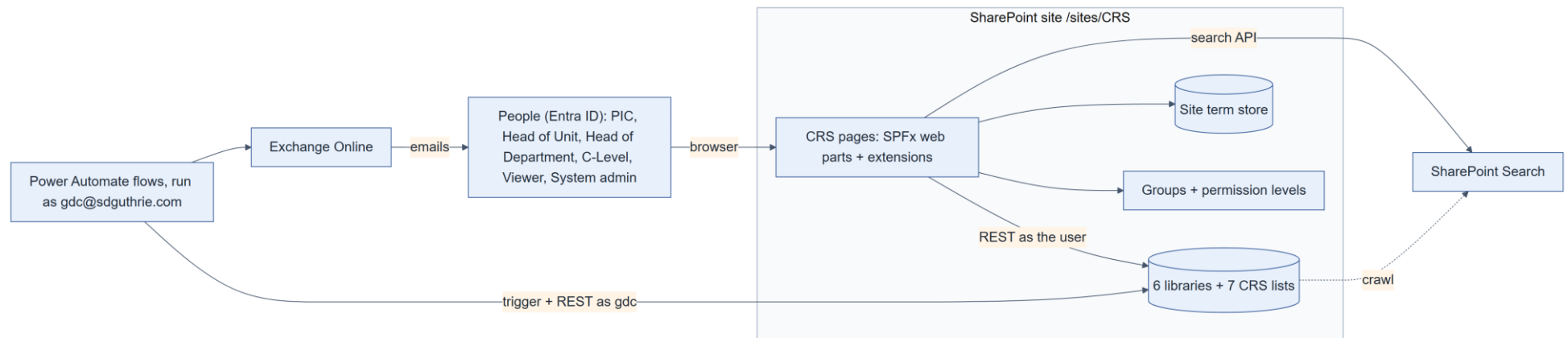


Figure 1. System context: people, the CRS site, flows and Microsoft 365 services

2.4 Design principles

- **Act as the user.** Web parts call SharePoint REST with the signed-in user's own permissions. There is no backend server, no Microsoft Graph and no app-only permission.
- **Proxy what users must not do.** Writing metadata (for uploaders), moving approved files, deleting, revoking shares and sending email are done by flows running as gdc.
- **Configuration in lists, not code.** Folder levels, codes, role mappings, file types and switches live in CRS lists and take effect on the next page load.
- **Names are resolved at run time.** The client renames lists, libraries and pages. Code resolves them from candidate titles and URL segments, never from literals.
- **Terms are keyed by GUID.** Labels can change (and some are corrupted on this tenant); GUIDs are the join keys everywhere.
- **Empty is not unknown.** A failed read is unknown, never an empty result. Form gates fail open; security and destructive decisions fail closed.
- **Permissions are additive.** Reconciliation only adds grants; deleting a group is the only revoke.

3. Architecture

3.1 Technology stack

Table 3. Technology stack

Layer	Technology	Version / note
UI framework	SharePoint Framework (SPFx)	1.23.0
Language	TypeScript	~5.8
UI library	React	17.0.1
Components	Fluent UI React	8.x
Build	Heft (rushstack)	npm run build; gulp is listed in package.json but unused
Runtime	Node.js	>= 22.14, < 23
Data access	SPHttpClient (SharePoint REST)	/_api, term store /_api/v2.1/termStore, Search /_api/search/query. @pnp/sp is installed but not used.
Automation	Power Automate (cloud flows)	25 flows, owner gdc@sdguthrie.com, default environment
Connectors	SharePoint, Office 365 Outlook	Connections named GDC Proxy - SharePoint and GDC Proxy - Outlook
Hosting	SharePoint Online site collection app catalog	/sites/CRS/AppCatalog; assets packaged inside the .sppkg (no CDN)
Tests	Jest via Heft	Pure logic modules have a x.test.ts; npx heft test -clean

3.2 Logical architecture

CRS has no server of its own. The browser runs the SPFx code inside SharePoint pages and reads and writes SharePoint directly as the signed-in user. Power Automate is the only server-side component; it works on the same lists and libraries as the service account.

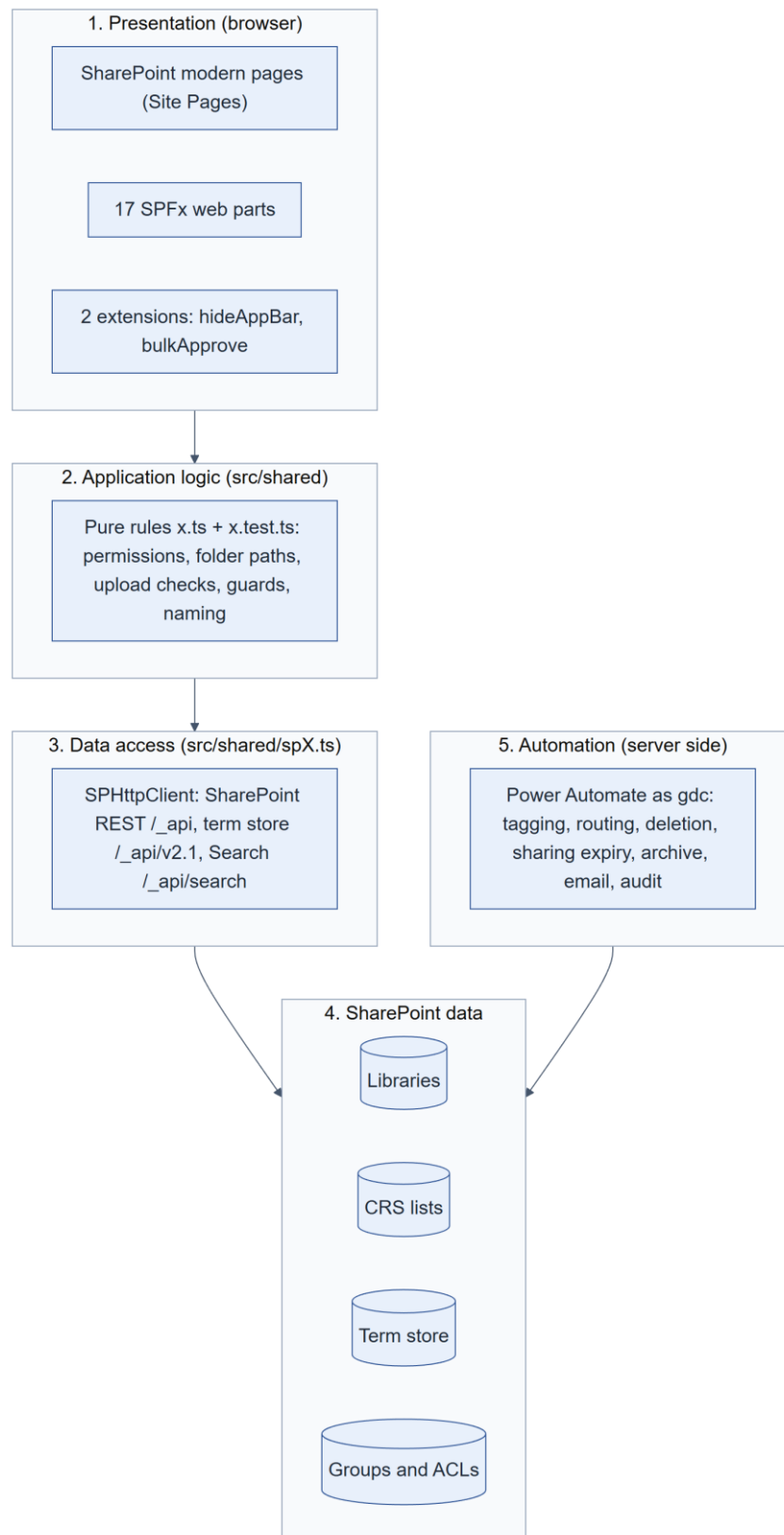


Figure 2. Logical layers

- **Presentation:** one web part per page; the extensions change the library command bar.
- **Application logic:** `src/shared/*.ts`. Rules are pure functions with tests (permissions, folder paths, upload checks, approval guards, request scope). Behaviour changes are made here.
- **Data access:** `src/shared/sp*.ts` and the REST calls inside components. Every read that decides an action is cache-busted; throttled calls (429/503) are retried by `throttleRetry.ts`.

- **Data:** SharePoint libraries, lists, term store and permissions (chapters 4 and 5).
- **Automation:** flows triggered by list changes or schedules (chapter 8).

3.3 Deployment architecture

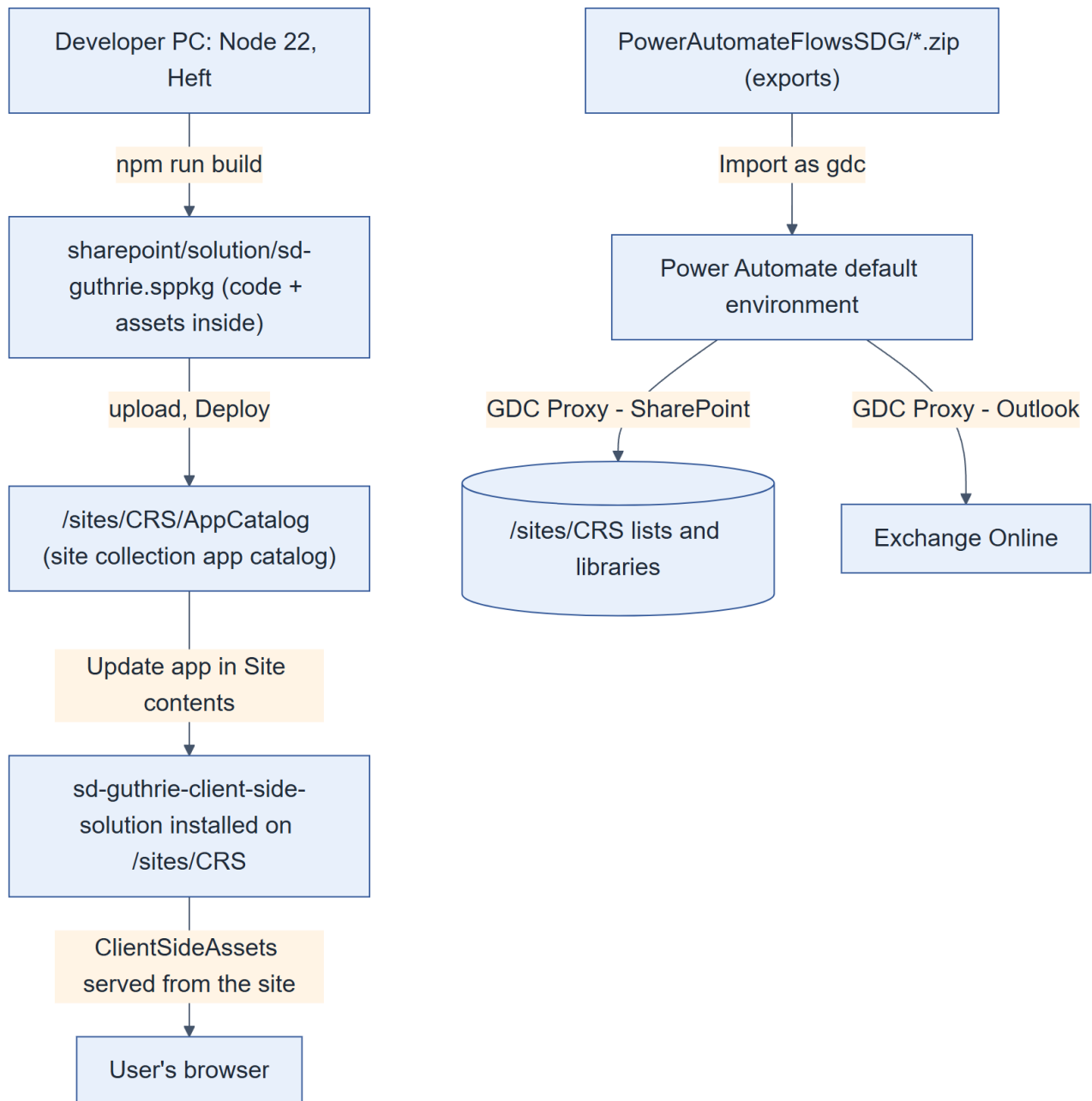


Figure 3. Deployment view

Table 4. Deployment facts

Item	Value
Solution name / id	sd-guthrie-client-side-solution / 4c93b50e-03c1-4005-a280-add0da86c86f (never change the id)
Package	sharepoint/solution/sd-guthrie.sppkg, includeClientSideAssets: true (code ships inside the package)
App catalog	Site collection app catalog /sites/CRS/AppCatalog. The tenant app catalog is not used.
API permissions	None (webApiPermissionRequests is empty): no admin consent is needed.

Item	Value
Feature deployment	skipFeatureDeployment: false. On this tenant the extensions did not register from the feature and were added by hand (Appendix A.5).
Flow environment	SD Guthrie default environment, signed in as gdc.

3.4 Solution components

Table 5. Components in sd-guthrie.sppkg

Component	Type	Id	Used on page
Upload Form (form)	Web part	21b82fca-081a-4ce4-bbe6-a83f2fce63a3	Upload-Form.aspx
Bulk Upload	Web part	299700f0-9a8e-4367-8018-1f9d54b92705	Bulk-Upload.aspx
Approval Document	Web part	3a8f2e6d-1b4c-4a7f-8e9d-2c5b7f3a1e6d	Approval-Document.aspx
My Submissions	Web part	5c9d1a83-6f24-4b07-9e51-8d3a0c7f2b46	My-Submissions.aspx, Document-Viewer.aspx
GDC Requests	Web part	7a4f1e93-2c58-4d07-b6a1-9e3c85f2d410	CRS-Request.aspx
GDC Search	Web part	8b2f4a95-7c31-4d68-9e04-1a6b3f8c25d7	Home.aspx
Folder Administration	Web part	02007994-7d63-4178-9a43-2b6ec96f9e44	Folder-Administrator.aspx
Folder Structure	Web part	ba145053-3728-4b6c-8a1e-a393c520abb2	Used inside the Folder Administration guided flows
Group Management	Web part	3f81c6d2-45ae-4b93-8c17-0d5e2a9f6b41	Group-Management.aspx
Page Access	Web part	2d6d7010-8bc0-4ac2-8573-fe694db220f8	Page-Access.aspx
Approval Library Access	Web part (signpost)	591eb30f-4bb3-4853-a799-a41c5664d7fc	Approval-Library-Access.aspx
Site Access	Web part (signpost)	695640a1-5bf9-4582-adbe-30a988c4945a	Site-Access.aspx
Folder Access	Web part (retired signpost)	a5955b3e-0945-4ad4-9cb1-db8006c7fa7f	Folder-Access.aspx
GDC Settings	Web part	4d8b7e21-9c65-4a3f-b028-5e71d9c4a836	CRS-Settings.aspx
GDC Configuration	Web part	c7a41f6b-5d92-4e08-9a37-6b1f2c8d4e05	CRS-Configuration.aspx
GDC Audit Log	Web part	9e2c4d17-8b3a-4f60-a5d1-7c0e6b28f394	CRS-Audit-Log.aspx
Folder Onboarding	Web part (legacy)	7b3e9c14-2a8d-4f61-9b0e-5d3c8a1f4e72	Not on any page
hideAppBar	Application customizer	c1d2e3f4-a5b6-47c8-d9e0-f1a2b3c4d5e6	Every CRS library (registered as CRS New Folder Button)
bulkApprove	List view command set	6d1a9f47-3c82-4e15-b7a0-9f2e5c8d4136	Document libraries (registered as CRS Bulk Approve, RegistrationId 101)
uploadCommand	Command set (legacy)	c2fd5b59-02ed-4892-87f2-2418fefb4980	Not registered; links to an old sandbox. Ignore.

3.5 Pages

Pages are modern Site Pages, each holding one CRS web part. The client renames pages, so the settings directory finds them by file name at run time. Read from the live site:

Table 6. Site Pages with web parts

File	Page title	Web parts on the page
Home.aspx	Home	CRS Search, Document library, Document library
Approval-Documents.aspx	Approval Document	Approval Document
CRS-Settings.aspx	 GDC Settings	CRS Settings
Upload-Form.aspx	Document Upload	form
Folder-Administrator.aspx	Folder Management	Folder Administration
Group-Management.aspx	Group Management	Group Management
Folder-Access.aspx	Folder Access	Title area, Folder Access
Site-Access.aspx	Site Access	Site Access
Approval-Library-Access.aspx	Approval Document Access	Approval Library Access
Page-Access.aspx	Page Access	Page Access
CRS-Configuration.aspx	CRS Configuration	CRS Configuration
CRS-Audit-Log.aspx	CRS Audit Log	CRS Audit Log
Bulk-Upload.aspx	Bulk Upload	Bulk Upload
My-Submissions.aspx	 My Submission	My Submissions
CRS-Request.aspx	File Permission	CRS Requests
Page.aspx		Title area
Document-Viewer.aspx	Document-Viewer	My Submissions
User-Guides.aspx	User Guides	Image, Image, Image, Image
User-Guideline-Viewer.aspx	User Guideline Viewer	Quick links, File and media
User-Guide-Uploader-Approver.aspx	User Guideline Uploader Approver	Quick links, File and media
User-Guide-Viewer.aspx	User Guide Viewer	Quick links, File and media
User-Guide(1).aspx	User Guide	Quick links, File and media
User-Guide.aspx	User Guide	Quick links, File and media

3.6 Code organisation

Table 7. Repository layout

Folder	Contents
src/webparts/<name>/	One folder per web part: <Name>WebPart.ts (entry, property pane, render) and components/<Name>.tsx (React UI).
src/extensions/	hideAppBar (adds + New Folder; hides native Upload, Approve/Reject and Share), bulkApprove (Approve or reject panel), uploadCommand (legacy).
src/shared/	All shared logic. Pure rules in x.ts with x.test.ts; SharePoint calls in spX.ts. About 120 modules.
config/	package-solution.json (version, component ids), serve.json (local debug pages).
scripts/	Browser-console diagnostics and repairs; Python flow generators.
PowerAutomateFlowsSDG/	Exported flow packages. Re-export and commit after every flow change.
sharepoint/assets/elements.xml	Feature definition for the extensions (did not provision on this tenant).

Table 8. Shared module catalogue

Area	Shared modules
Names of lists, libraries, groups	naming.ts, spNaming.ts
Folder structure	folderChain.ts, folderAbbreviation.ts, abbreviationDraft.ts, spAbbreviationTree.ts, folderFullName.ts, termChains.ts, tierColumnGuard.ts
Folder Map and permission probes	dmsFolderMap.ts, folderMapDuplicates.ts, segmentReadiness.ts
Guided admin flows	folderFlows.ts
Segments and migration	newSegment.ts, segmentDeletion.ts, segmentRecode.ts, spSegmentRecode.ts, reconScope.ts, subtreeMigration.ts, uploadPause.ts
Groups and roles	groupMapModel.ts (PERSONAS), bulkGroups.ts, groupMappings.ts, spGroups.ts, spGroupsFilter.ts, siteEntryGroup.ts, userAccess.ts, groupExportCsv.ts
Page access	pageAccessPolicy.ts, pageGrants.ts, adminPages.ts, grantSkip.ts
Upload	formModel.ts, uploadBatches.ts, allowedFileTypes.ts, fileTypeSettings.ts, inputSanitize.ts, optionalColumns.ts, spColumns.ts, legalPrivilege.ts, confidentialityOrder.ts, clashProxyDeletion.ts
Highly Confidential	hcRouting.ts, hcClearance.ts
Approval	approvalGuards.ts, approvalDestination.ts, approvalQueue.ts, selfApprove.ts
Submissions	mySubmissions.ts, submissionGroups.ts, submissionRecords.ts, spSubmissionRecords.ts
Requests	requests.ts, directFileActions.ts, deletionGuard.ts, liveRefresh.ts
Search	documentSearch.ts
Audit	auditLog.ts, spAuditLog.ts, displayName.ts
Network	throttleRetry.ts, networkErrors.ts, pathEncoding.ts
UI helpers	toast.tsx, pagination.tsx, filterSelect.tsx, datePicker.tsx, responsive.ts, fileDetailPanel.tsx, filePreview.ts and others

3.7 Runtime behaviour common to all web parts

- **Name priming.** On load, primeNames (in naming.ts) asks SharePoint which lists exist and resolves each logical list (LIST_SUFFIX: Config, Group Map, Folder Map, Term Abbreviation, Submissions, Requests, Audit Log) to CRS <suffix>, falling back to the old DMS <suffix>. Libraries resolve from candidate title arrays (LIBRARY_CANDIDATES, DOCUMENTS_CANDIDATES, HC_APPROVAL_CANDIDATES ...) and URL segments.
- **Config load.** CRS Config is read once per page load (settings rows and mode rows).
- **Permission probes.** Before offering an action, the code asks SharePoint for the user's effective permissions on the exact folder or item (EffectiveBasePermissions). Group Map membership is a second source for display; writes are always checked by SharePoint itself.
- **Cache-busting.** Reads that decide an action add Cache-Control: no-cache and a unique query parameter.
- **Optional columns.** A \$select naming an unknown column fails the whole read, so reads of optional columns use a fallback ladder (one rung per optional column).
- **Throttling.** HTTP 429/503 are retried with back-off (throttleRetry.ts).
- **No browser storage.** CRS keeps no state in localStorage or sessionStorage; all state lives in SharePoint.
- **Mobile.** Layouts are phone-first and widen with min-width media queries.

4. Data design

CRS has no database of its own. All data lives in SharePoint Online on the CRS site: document libraries hold the files and their metadata, custom lists hold configuration and records, the site term store holds the classification, and SharePoint groups and permissions hold security. This chapter was produced from a read-only capture of the live site on 2026-10-01.

4.1 Where data is stored

Table 9. Data stores

Store	What it holds	Written by	Retention
Approval libraries (2)	Uploaded files waiting for approval, rejected files, and their metadata	Upload Form, Bulk Upload, tagging flow, approvers	Until routed (approved) or deleted; rejected files stay
Approved libraries (2)	Approved documents with version history	Auto-route flows only	7 years from creation, then archived
Archive libraries (2)	Documents older than 7 years, read-only	Archive flows only	Indefinite
CRS Config	Settings and per-segment folder structure	Admin pages	Configuration
CRS Term Abbreviation	Term → folder code	Folder Administration	Configuration
CRS Group Map	Which group holds which role at which term	Group Management	Configuration
CRS Folder Map	Term → folder id and URL	Folder Reconciliation	Rebuilt by reconciliation
CRS Submissions	One record per uploaded file; survives deletion; carries TagPayload	Upload Form, Bulk Upload, flows	Indefinite (history)
CRS Requests	Delete and share requests and decisions	My Submissions, Requests page, flows	Indefinite
CRS Audit Log	Append-only event log	Web parts (<code>writeAudit</code>) and flows	Indefinite
Site term store	Segments, departments, units, Year, Document Type, Confidentiality Level	Admins in the term store tool	Configuration
SharePoint groups and ACLs	Who can open which library, folder and page	Group Management, Reconciliation, Requests page	Configuration
Site Pages	The CRS pages and their web part instances	Admins	Configuration
SharePoint Search index	Crawled copy of approved and archive libraries for Search	SharePoint crawler (re-crawl nudged every 6 h)	Follows the libraries
Power Automate	Flow definitions, connections, 28-day run history	gdc	Run history 28 days

4.2 Logical data model

The term GUID is the main key: it ties the term store to folder codes, folders and role grants. On the document side, `SubmissionFileId` ties a document to its submission record through every move (a file's `UniquelId` changes when Auto-route copies it), and `ItemUniquelId` ties requests and audit rows to a file.

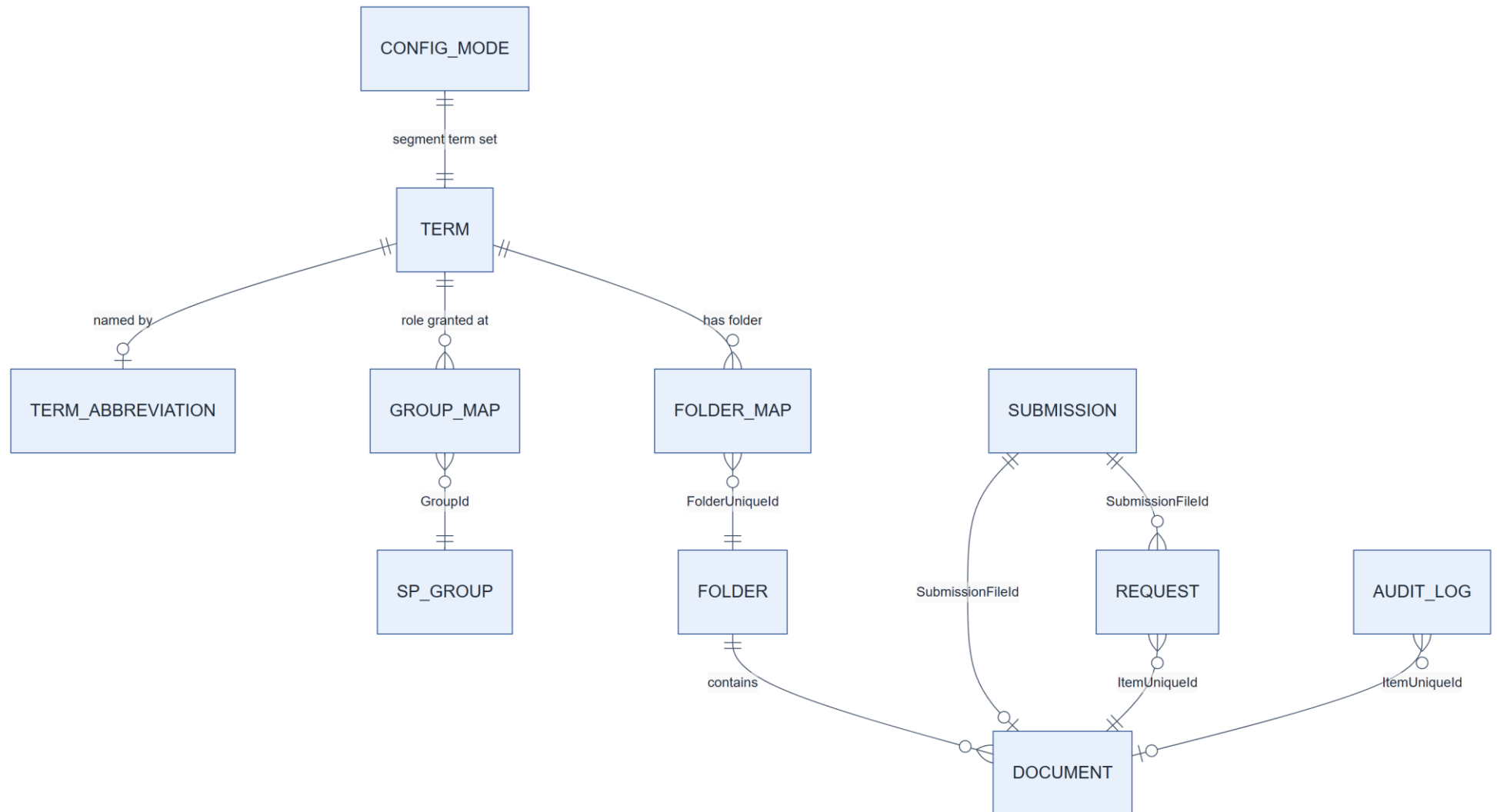


Figure 4. Logical data model (keys only)

Table 10. Keys and joins

Key	Format	Links
Term GUID	GUID from the term store	Term Abbreviation.TermGuid, Folder Map.TermGuid, Group Map.UnitTermGuid, document *Tid columns, Requests.UnitTermGuid; Config mode row TermSetGuid (set level)
Group id	SharePoint group Id (integer, stored as text)	Group Map.GroupId → site group
Folder UniqueId	GUID	Folder Map.FolderUniqueId → folder in the approval library
SubmissionRef / SubmissionId	SUB-yyyymmdd-xxxx	One upload (all its sets). Record.SubmissionRef = document.SubmissionId
BatchRef / BatchId	BAT-yyyymmdd-xxxx	One set (one destination folder)
SubmissionFileId	Generated per file by the form	Record ↔ document (approval and approved copies) ↔ Requests.SubmissionFileId
ItemUniqueId	File UniqueId	Requests and Audit Log → the file at the time of the event

4.3 Libraries

Table 11. Document libraries (live settings)

Library (title / URL)	Key in code	Role	List GUID	Content approval	Versioning	Items*	Unique perms
Approval for Document /ApprovalDocument	Staging	Pending and rejected uploads (normal)	eeb1bb19-ec53-4c41-8dc9-ecf255979c9b	Yes (Author + approvers)	Major, keep 500	175	Yes
Restricted & Confidential Document /Shared Documents	Documents	Approved documents (normal)	fb062dd-fcd0-4458-9161-2bfc19c917fa	No	Major, keep 500	175	Yes
Approval for Highly Confidential Document /HCAApprovalDocument	StagingHC	Pending and rejected uploads (Highly Confidential)	d935aa6d-dc48-4832-ba7e-eca485ecdff3	Yes (Author + approvers)	Major, keep 500	175	Yes
Highly Confidential Document /HCDocuments	DocumentsHC	Approved Highly Confidential documents	122a4aa9-65fb-479a-90a7-3866d19f51b4	No	Major, keep 500	175	Yes
Archive Restricted & Confidential Document /Archive	Archive	Approved documents older than 7 years (read-only)	647fd644-ab00-4358-9b3f-49031083fcf0	No	Major, keep 500	175	Yes
Archive Highly Confidential Document /HCArchive	ArchiveHC	Highly Confidential documents older than 7 years (read-only)	fc301054-abf5-4e53-957a-6f8c58a410b8	No	Major, keep 500	175	Yes

*Item counts include folders. At capture the libraries held only the folder tree (the launch purge emptied them on 30 September 2026).

The client renames library titles; the URL segment never changes. Code must resolve titles at run time (naming.ts), and the hideAppBar extension keeps its own title list (CRS_LIBRARIES). Add a renamed title to both, build and deploy.

Table 12. Content types per library

Library	Content types
Approval for Document	Document, Folder, CRS Folder
Restricted & Confidential Document	Document, Folder, CRS Folder
Approval for Highly Confidential Document	Document, Folder, CRS Folder
Highly Confidential Document	Document, Folder, CRS Folder
Archive Restricted & Confidential Document	Document, Folder, CRS Folder
Archive Highly Confidential Document	Document, Folder, CRS Folder

CRS Folder is the folder content type that carries the Full Name column.

Table 13. Library views (non-hidden)

Library	View	Columns	Filter / sort
Approval for Document	All Documents	DocIcon, LinkFilename, Author, Created, _ModerationStatus, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, Full_x0020_Name, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	FileLeafRef
Approval for Document	Pending Files (default)	DocIcon, LinkFilename, Author, Created, _ModerationStatus, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, Full_x0020_Name, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay, Keyword, NextReminderAt	Eq _ModerationStatus = Pending
Approval for Document	Rejected Files	DocIcon, LinkFilename, Author, Created, _ModerationStatus, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, Full_x0020_Name, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	Eq _ModerationStatus = Rejected
Restricted & Confidential Document	All Documents (default)	DocIcon, LinkFilename, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	FileLeafRef
Restricted & Confidential Document	Show All Files	DocIcon, LinkFilename, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay, Keyword	—

Library	View	Columns	Filter / sort
Approval for Highly Confidential Document	All Documents	DocIdcon, LinkFilename, Editor, Full_x0020_Name, _ModerationStatus, Business_x0020_Segment, Department, Unit, SubUnit, DocumentDate, Vendor_x002f_CustomerName, ProjectName, LegallyPrivileged, Remark, Document_x0020_Type, Year, Confidentiality_x0020_Level, Created, FileSizeDisplay	FileLeafRef
Approval for Highly Confidential Document	Pending Files (default)	DocIdcon, LinkFilename, Editor, Full_x0020_Name, _ModerationStatus, Business_x0020_Segment, Department, Unit, SubUnit, DocumentDate, Vendor_x002f_CustomerName, ProjectName, LegallyPrivileged, Document_x0020_Type, Year, Confidentiality_x0020_Level, Created, FileSizeDisplay, Remark, Author, NextReminderAt	Eq _ModerationStatus = Pending
Approval for Highly Confidential Document	Rejected Files	DocIdcon, LinkFilename, Editor, Full_x0020_Name, _ModerationStatus, Business_x0020_Segment, Department, Unit, SubUnit, DocumentDate, Vendor_x002f_CustomerName, ProjectName, LegallyPrivileged, Document_x0020_Type, Year, Confidentiality_x0020_Level, Created, FileSizeDisplay, Remark	Eq _ModerationStatus = Rejected
Highly Confidential Document	All Documents (default)	DocIdcon, LinkFilename, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	FileLeafRef
Highly Confidential Document	Show All Files	DocIdcon, LinkFilename, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	—
Archive Restricted & Confidential Document	All Documents (default)	DocIdcon, LinkFilename, Archived, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, FullName, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	FileLeafRef
Archive Restricted & Confidential Document	View All Files	DocIdcon, LinkFilename, Archived, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, Full_x0020_Name, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	—
Archive Highly Confidential Document	All Documents (default)	DocIdcon, LinkFilename, Archived, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, FullName, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	FileLeafRef
Archive Highly Confidential Document	View All Files	DocIdcon, LinkFilename, Archived, Author, Created, Confidentiality_x0020_Level, LegallyPrivileged, Business_x0020_Segment, Department, Unit, SubUnit, Year, Document_x0020_Type, FullName, ProjectName, Vendor_x002f_CustomerName, DocumentDate, SubmissionId, Editor, FileSizeDisplay	—

The bundled approver emails link to the Pending Files view of each approval library. Do not rename or delete those views.

4.4 Document columns

All six libraries share one set of document columns, so a file keeps its metadata when it is routed and archived. The table lists every visible custom column found on the live libraries that CRS uses.

Table 14. Document columns

Display name	Internal name	Type	Libraries	Meaning	Written by
Title	Title	Text	All 6	SharePoint's built-in title. Not used by the logic.	—
Description	_ExtendedDescription	Multi-line text	All 6	Built-in document Description. Shown in the details panel when filled.	Users (optional)
Business Segment	Business_x0020_Segment	Text	All 6	Business segment label (e.g. Group Head Office).	Apply pending tags (from TagPayload); copied by Auto-route
BusinessSegmentTid	BusinessSegmentTid	Text	All 6	Business segment term GUID. Logic compares GUIDs, never labels.	Apply pending tags; copied by Auto-route
Department	Department	Text	All 6	Permissioned tier 1 label.	Apply pending tags
DepartmentTid	DepartmentTid	Text	All 6	Tier 1 term GUID.	Apply pending tags
Unit	Unit	Text	All 6	Permissioned tier 2 label (the confidentiality boundary).	Apply pending tags
UnitTid	UnitTid	Text	All 6	Unit term GUID. Flows use it to find the unit's approver group in CRS Group Map.	Apply pending tags
SubUnit	SubUnit	Text	All 6	Optional per-unit level label.	Apply pending tags
SubUnitTid	SubUnitTid	Text	All 6	Sub Unit term GUID.	Apply pending tags
Document Date	DocumentDate	Date and time	All 6	Document date. Part of the file name.	Apply pending tags (from the upload form)
Vendor/CustomerName	Vendor_x002f_CustomerName	Text	All 6	Vendor or customer. Optional part of the file name.	Apply pending tags
ProjectName	ProjectName	Text	All 6	Project. Optional part of the file name.	Apply pending tags
Remark	Remark	Text	All 6	Remark for the approver. Removed from the upload form on 21 Sep 2026 (client); kept for older files.	Older uploads only

Display name	Internal name	Type	Libraries	Meaning	Written by
LegallyPrivileged	LegallyPrivileged	Yes/No	All 6	Legally privileged flag. Asked only for the levels in the legallyPrivilegedFor config row.	Apply pending tags
Full Name	Full_x0020_Name	Text	All 6	On folders (CRS Folder content type): the full label behind the abbreviated folder name.	Folder Reconciliation
Document Type	Document_x0020_Type	Managed metadata (Document Type)	All 6	Managed metadata, Document Type term set.	Apply pending tags; copied explicitly by Auto-route
Year	Year	Managed metadata (Year)	All 6	Managed metadata, Year term set.	Apply pending tags; copied explicitly by Auto-route
Confidentiality Level	Confidentiality_x0020_Level	Managed metadata (Confidentiality Level)	All 6	Managed metadata, Confidentiality Level term set. hcConfidentialityLevel names the HC value.	Apply pending tags; copied explicitly by Auto-route
Submission Id	SubmissionId	Text	All 6	Upload id (SUB-yyyymmdd-xxxx), same as SubmissionRef on the record.	Apply pending tags; copied by Auto-route
Batch Id	BatchId	Text	All 6	Batch (set) id (BAT-yyyymmdd-xxxx).	Apply pending tags; copied by Auto-route
Bulk Import	BulkImport	Yes/No	All 6	True for Bulk Upload files. Triggers auto-approval and suppresses the approved email. Must stay visible.	Bulk Upload
Archived	Archived	Yes/No	All 6	Set by the seven-year archive mover. Used for badges in My Submissions and Search; the library, not this flag, decides access.	Archive flows
Submission File Id	SubmissionFileId	Text	All 6	Per-file id. The join key to CRS Submissions; survives routing (Uniqueld does not).	Apply pending tags; copied by Auto-route
Approved By	ApprovedBy	Text	All 6	Approver's email, written in its own MERGE before the status change (SharePoint keeps no approved-by field).	Approval Document page, Approve or reject panel, self-approve
Keyword	Keyword	Text	All 6	Free-text keywords for search.	Apply pending tags

Display name	Internal name	Type	Libraries	Meaning	Written by
Image Tags	MediaServiceImageTags	Managed metadata (Image Tags)	Approval, Documents, HC approval, HC documents	SharePoint's automatic image tags. Not used by CRS.	SharePoint
Approval Comment	ApprovalComment	Multi-line text	All 6	Comment typed when approving. The rejection reason is the built-in moderation comment.	Approval Document page, panel
NextReminderAt	NextReminderAt	Date and time	Approval, HC approval	When the next approver reminder is due. Approval libraries only.	Reminder flows
Full Name	FullName	Text	Archive, HC archive	Second Full Name column that exists only on the two archive libraries.	Archive flows

Built-in columns the system relies on

Table 15. Built-in columns used

Column (REST name)	Use in CRS
OData__ModerationStatus (CAML _ModerationStatus)	Approval state: 0 Approved, 1 Rejected, 2 Pending. Never MERGE it together with another field (HTTP 500).
OData__ModerationComments	Rejection reason, emailed to the uploader.
Author (Created By)	The uploader. My Submissions filters on AuthorId. Auto-route re-stamps it on the approved copy. Never use Editor as the uploader.
Editor (Modified By)	Re-stamped to gdc by Auto-route; not reliable for who approved.
Created	Upload date; the archive flows use it for the 7-year cut-off.
UniqueId, FileRef, FileLeafRef	File identity and path. UniqueId changes when Auto-route copies a file.
FileSystemObjectType / CAML FSObjType	0 file, 1 folder. Queues and flows exclude folders.
TaxCatchAll and the hidden note fields	SharePoint's storage for managed metadata values.

Folder-level columns created by Folder Structure

When an admin adds a below-Unit folder level, Folder Structure creates a label column and a ...Tid (term GUID) column on all six libraries. The live libraries carry the pairs below from client testing. At capture no CRS Config mode row referenced them (all three segments use Department › Unit › Year › Document Type), so they hold no live meaning. Remove one only after confirming that no mode row's Levels or PendingLevels names it and no document holds a value.

Table 16. Unused tier columns at capture

Display name	Internal name	Libraries
Mini	Mini	All 6
Mini ID	MiniTid	All 6
Showroom Branch	ShowroomBranch	All 6
Showroom Branch ID	ShowroomBranchTid	All 6
European Brand	EuropeanBrand	All 6
European Brand ID	EuropeanBrandTid	All 6
Japanese Brand	JapaneseBrand	All 6

Display name	Internal name	Libraries
Japanese Brand ID	JapaneseBrandTid	All 6
European Brand Test	EuropeanBrandTest	All 6
European Brand Test ID	EuropeanBrandTestTid	All 6
Japenese Brand Test	JapeneseBrandTest	All 6
Japenese Brand Test ID	JapeneseBrandTestTid	All 6
Test	Test	All 6
Test ID	TestTid	All 6
te	te	All 6
te ID	teTid	All 6
New Folder level testing	NewFolderleveltesting	All 6
New Folder level testing ID	NewFolderleveltestingTid	All 6
Volvo	Volvo	All 6
Volvo ID	VolvoTid	All 6
volvo	volvo0	All 6
volvo ID	volvoTid0	All 6
Branch Test	BranchTest	All 6
Branch Test ID	BranchTestTid	All 6
Deparment	Deparment	All 6
Deparment ID	DeparmentTid	All 6
Tset again	Tsetagain	All 6
Tset again ID	TsetagainTid	All 6
test again1	testagain1	All 6
test again1 ID	testagain1Tid	All 6
Do	Do	All 6
Do ID	DoTid	All 6
t	t	All 6
t ID	tTid	All 6
d	d	All 6
d ID	dTid	All 6
y	y	All 6
y ID	yTid	All 6
sa	sa	All 6
sa ID	saTid	All 6
asad	asad	All 6
asad ID	asadTid	All 6
sad	sad	All 6
sad ID	sadTid	All 6

4.5 CRS lists

Table 17. CRS lists (live)

List	URL	List GUID	Rows	Unique perms
CRS Audit Log	/sites/CRS/Lists/CRS Audit Log	4ac219e5-b855-46a3-be4b-4e96bba546c5	298	No
CRS Config	/sites/CRS/Lists/CRS Config	22e4f613-1693-407c-9f94-318088c51775	16	No
CRS Folder Map	/sites/CRS/Lists/CRS Folder Map	8a0c644d-638a-4282-83bd-6c73a9e36618	172	No
CRS Group Map	/sites/CRS/Lists/CRS Group Map	5acf3f3e-d955-4a82-8a49-24db3a353514	1708	No
CRS Requests	/sites/CRS/Lists/CRS Requests	1bcb1aa1-fbf1-4d1b-a6d2-88436824c297	0	Yes
CRS Submissions	/sites/CRS/Lists/CRS Submissions	dd1bc5bd-daa7-491a-9dc7-b81212c391ab	0	Yes
CRS Term Abbreviation	/sites/CRS/Lists/CRS Term Abbreviation	b1c3d460-18da-466f-a19c-5f597d113cfa	172	No

The lists are found by suffix with a CRS or DMS prefix, never by a literal title. CRS Requests, CRS Audit Log and CRS Submissions create themselves and their columns when their web part first opens, and the Requests page has an add-missing-columns repair. Never hand-create a column on them: SharePoint fixes a column's internal name at creation.

Table 18. CRS Config columns (16 rows at capture)

Internal name	Type	Indexed	Meaning
ConfigType	Text		setting or mode.
SettingValue	Text		Value of a setting row (text; yes/no settings use yes/no).
ModeLabel	Text		Mode row: the segment's display name.
Category	Text		Mode row: BusinessSegment.
TermSetGuid	Text		Mode row: the segment's term set (departments, units).
StagingFolder	Text		Mode row: the segment's top folder code (GHO, MHO, NBPOLHO).
SortOrder	Number		Mode row: order in the segment picker.
Levels	Multi-line text		Mode row: JSON array of folder levels (see 4.6).
PendingLevels	Multi-line text		Mode row: a staged structure change waiting for migration.
AllowedFileTypes	Choice (multi)		Multi-choice of extensions on the allowedExtensions row.

Table 19. CRS Term Abbreviation columns (172 rows at capture)

Internal name	Type	Indexed	Meaning
TermGuid	Text		Term GUID (the key).
Abbreviation	Text		Short folder code, unique among siblings (e.g. GIT).
Level	Text		Department or Unit (or a below-Unit level name).

Table 20. CRS Folder Map columns (172 rows at capture)

Internal name	Type	Indexed	Meaning
TermGuid	Text		Unit (or department) term GUID.
FolderUniqueId	Text		Uniqueld of the folder in the approval library.
FolderUrl	Multi-line text		Server-relative URL of that folder.
Section	Text		Segment code (GHO, MHO, NBPOLHO).
Library	Text		Library the row points at (blank = approval library).

Table 21. CRS Group Map columns (1708 rows at capture)

Internal name	Type	Indexed	Meaning
GroupId	Text		SharePoint group id (text).
GroupName	Text		Group title at the time of writing.
Segment	Text		Segment term set GUID the row belongs to.
UnitTermGuid	Text		Term the role applies at (unit, department or segment term).
Role	Text		One role code (UPL, APR, SHARE, MEMBER ...). A group has one row per role.
Scope	Text		Folder for every row at handover.
Target	Text		Reserved; empty at handover.

Table 22. CRS Submissions columns (0 rows at capture)

Internal name	Type	Indexed	Meaning
SubmissionRef	Text		Upload id SUB-yyyyymmdd-xxxx.
BatchRef	Text		Set id BAT-yyyyymmdd-xxxx.
SubmissionFileId	Text		Per-file id stamped on the document. Join key.
ItemUniqueId	Text		Uniqueld of the file in the approval library (changes when routed).
FileName	Text		File name at upload.
ItemPath	Multi-line text		Server-relative path at upload.
LibraryTitle	Text		Library title at upload.
UploadedBy	Text		Uploader's email.
UploadedAt	Date and time		Upload time (UTC).
MetadataSnapshot	Multi-line text		JSON of the metadata LABELS at upload, for history.
Source	Text		Which screen wrote it (upload form or bulk upload).
ReplacedAt	Date and time		Set by Auto-route when a newer upload replaced this file.
ReplacedBy	Text		Who replaced it.
ArchivedAt	Date and time		Set by the archive flows.
WithdrawnAt	Date and time		Set when the uploader cancelled (direct staging delete).
WithdrawnBy	Text		Who withdrew it.
TagPayload	Multi-line text		JSON of the field values the tagging flow must write (internal names).
TagStatus	Text		Blank = waiting, Tagged, or Failed.

Internal name	Type	Indexed	Meaning
TagError	Multi-line text		Error text when tagging failed.

ApproverNotifiedAt (Date and Time, include time) is not yet on the live CRS Submissions list. It must be added before the bundled approver email is turned on (chapter 12).

Table 23. CRS Requests columns (0 rows at capture)

Internal name	Type	Indexed	Meaning
RequestType	Text		Deletion or Share.
Stage	Text		pending or approved (stage of the target file).
Status	Text		Pending, Approved, Rejected, Failed, Cancelled, Revoked.
ItemUniqueId	Text		Uniqueld of the target file.
ItemName	Text		File name.
ItemUrl	Text		Server-relative URL of the file.
Segment	Text		Segment of the file.
Unit	Text		Unit label.
UnitTermGuid	Text		Unit term GUID; decides which approvers see the request.
RequestedBy	Text		Requester's email.
RequestedAt	Date and time		When raised.
Reason	Multi-line text		Requester's reason.
ShareWith	Text		Share: recipient email(s).
SharePermission	Text		Share: View or Edit.
ExpiresAt	Date and time		Share: expiry. Blank = permanent.
DecidedBy	Text		Approver's email.
DecidedAt	Date and time		Decision time.
DecisionNote	Multi-line text		Approver's reason, plus notes appended by flows.
SubmissionFileId	Text		Join key to CRS Submissions (replaced-file guard).
RevokedBy	Text		Who revoked a share (gdc for expiry).
TargetETag	Text		File ETag when the delete was decided; the delete flow refuses if it changed.

Table 24. CRS Audit Log columns (298 rows at capture)

Internal name	Type	Indexed	Meaning
EventTime	Date and time	Yes	When it happened (UTC).
EventType	Text	Yes	Event code (Text, never Choice). See 4.10.
Outcome	Text		Success or failure text.
ActorName	Text		Who did it (display name).
ActorEmail	Text	Yes	Who did it (email).
Source	Text		Screen or flow that wrote the row.
LibraryName	Text		Library involved.
ItemUniqueId	Text	Yes	File or folder Uniqueld.

Internal name	Type	Indexed	Meaning
ItemName	Text		File or folder name.
ItemPath	Multi-line text		Path.
Segment	Text		Segment.
UnitPath	Text		Segment / department / unit codes.
Details	Multi-line text		JSON or text with event details.

4.6 Configuration data

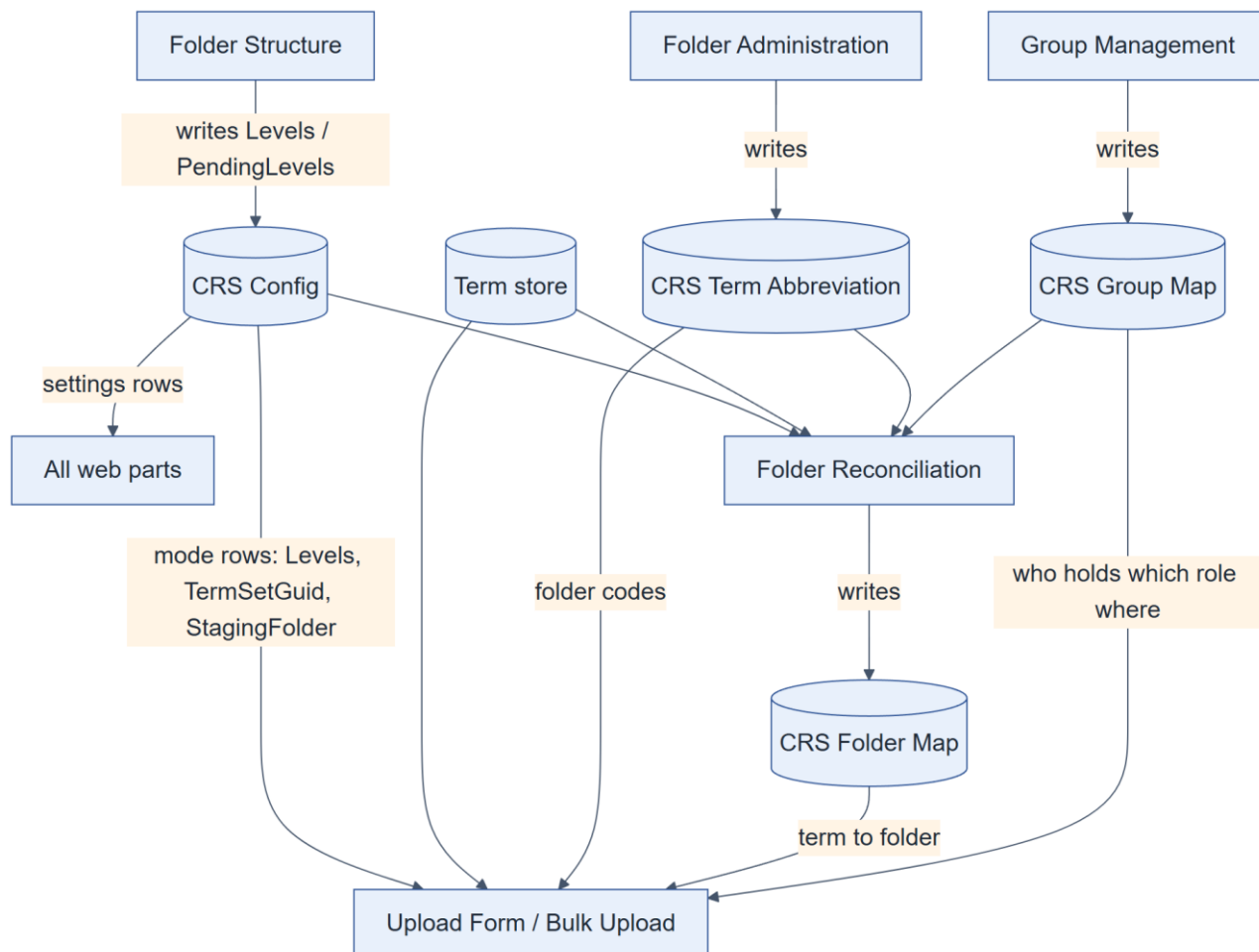


Figure 5. How the configuration lists drive the system

Settings rows (live values)

Table 25. CRS Config settings

Row (Title)	Value	Effect
termSet_documentType	fd973a45-3c34-483a-bfae-2a5b9dc5a556	Term set for the Document Type picker.
termSet_yearPeriod	3a262838-fbfe-4219-84e6-f3d1d6710163	Term set for the Year picker.
termSet_confidentiality	7901b1dd-a04b-4102-b1ce-dfe169e6248d	Term set for the Confidentiality Level picker.
hcConfidentialityLevel	Highly Confidential	Which confidentiality label means Highly Confidential. Without it every uploader sees the HC option.

Row (Title)	Value	Effect
legallyPrivilegedFor	Confidential;Highly Confidential	Confidentiality levels for which the form asks Legally Privileged (semicolon list).
tenantDomains	sdguthrie.com	Domains that count as internal for share requests. (Two identical rows exist; harmless, keep one.)
allowExternalSharing	no	Whether a share request may name an outside address.
uploadsPaused	no	yes pauses all uploads (set during a folder migration).
allowedExtensions	.xls .xlsx .jpg .pdf .jpeg .png	Allowed upload file types (AllowedFileTypes). Nothing ticked blocks every upload, on purpose.
tenantDomains	sdguthrie.com	Domains that count as internal for share requests. (Two identical rows exist; harmless, keep one.)
autoApproveOwnUpload	yes	A Head of Unit's own upload approves itself after a live ApproveItems probe.
noApprovalEmailFor	crs@sdguthrie.com	Not read by the code or by any exported flow at handover (legacy row).
bulkUploadRestricted	no	Not read by the code or by any exported flow at handover (legacy row).

recon_departmentFanOut has no row on the live site; absent means on, which it must stay (department-level grants reach every unit folder below).

Mode rows: one per business segment

Table 26. CRS Config mode rows

Row	Segment	Top folder	Term set	Levels (* = permissioned)
mode_group_head_office	Group Head Office	GHO	271fa0cd-a1a8-4b13-9622-27b24038aa76	Department * › Unit * › Year › Document Type
mode_minamas_head_office	Minamas Head Office	MHO	d16dfefb-c542-48ae-906f-89c4f1982b9b	Department * › Unit * › Year › Document Type
mode_nbpol_head_office	NBPOL Head Office	NBPOLHO	21ee6b63-8601-4b0b-a470-0d68d35f112a	Department * › Unit * › Year › Document Type

Levels is a JSON array, one object per folder level below the segment, in order:

```
[{"label":"Department","column":"Department","labelCol":"Department","tidCol":"DepartmentTid","permissioned":true},
{"label":"Unit","column":"Unit","labelCol":"Unit","tidCol":"UnitTid","permissioned":true},
{"label":"Year","column":"Year","labelCol":"Year","termSet":"<Year term set>","permissioned":false},
{"label":"Document Type","column":"DocumentType","labelCol":"Document_x0020_Type","termSet":"<Document Type term set>","permissioned":false}]
```

- Exactly two levels are permissioned. Year and Document Type are fixed and come from their own term sets.
- labelCol / tidCol are the document columns that receive the label and term GUID.
- A missing permissioned key on the first two levels means permissioned (the MHO and NBPOLHO rows).
- A structure change on a segment that already holds documents is written to PendingLevels and applied by the migration (6.8).

4.7 Term store

Terms live in the site collection's own term group, managed at `/sites/CRS/_layouts/15/termstoremanager.aspx` (not the tenant admin centre). The code reads them through `/_api/v2.1/termStore/sets/{id}/children`.

Table 27. Term sets (live)

Term set	GUID	Terms	By depth	Used for
Document Type	fd973a45-3c34-483a-bfae-2a5b9dc5a556	23	L1: 23	Document Type column and folder level
Year	3a262838-fbfe-4219-84e6-f3d1d6710163	2	L1: 2	Year column and folder level
Confidentiality Level	7901b1dd-a04b-4102-b1ce-dfe169e6248d	3	L1: 3	Confidentiality column; decides HC routing
Group Head Office	271fa0cd-a1a8-4b13-9622-27b24038aa76	71	L1: 8, L2: 63	Segment folder tree: level 1 = departments, level 2 = units
Minamas Head Office	d16dfefb-c542-48ae-906f-89c4f1982b9b	69	L1: 20, L2: 49	Segment folder tree: level 1 = departments, level 2 = units
NBPOL Head Office	21ee6b63-8601-4b0b-a470-0d68d35f112a	33	L1: 12, L2: 20, L3: 1	Segment folder tree: level 1 = departments, level 2 = units

- Segment term sets: level 1 = departments (40 across the three segments), level 2 = units (132). One NBPOL term sits at level 3.
- Folder names come from CRS Term Abbreviation, keyed by term GUID, not from labels.
- **Rename terms; never delete and re-create them.** A new term gets a new GUID and orphans its Term Abbreviation, Folder Map and Group Map rows, and every document tagged with it.
- Some stored labels on this tenant are corrupted (show a number). Always compare by GUID.
- The tenant's Image Tags term set (1871 terms) belongs to SharePoint's image tagging and is not used by CRS.

4.8 Folder model

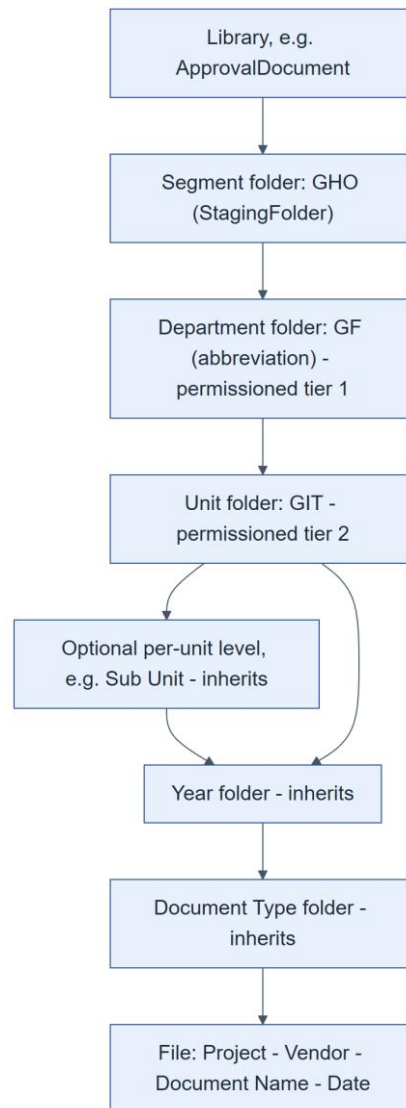


Figure 6. Folder tree and where permissions break

- Path: <Library>/<Segment>/<Department>/<Unit>/[Sub Unit]/<Year>/<Document Type>/<file>.
- Segment folder name = the mode row's StagingFolder. Department and unit folder names = abbreviations.
- Permissions are broken at the permissioned folders (segment, department, unit). Everything below inherits from the unit.
- The same tree exists in all six libraries. CRS Folder Map records the approval-library folders (172 rows at capture: GHO 71, MHO 69, NBPOLHO 32).
- File name = [Project] - [Vendor] - [Document Name] - [Date]; blank optional parts are dropped.
- Below-Unit folders are created on demand by the upload form and by Auto-route.

4.9 Record lifecycles

Document

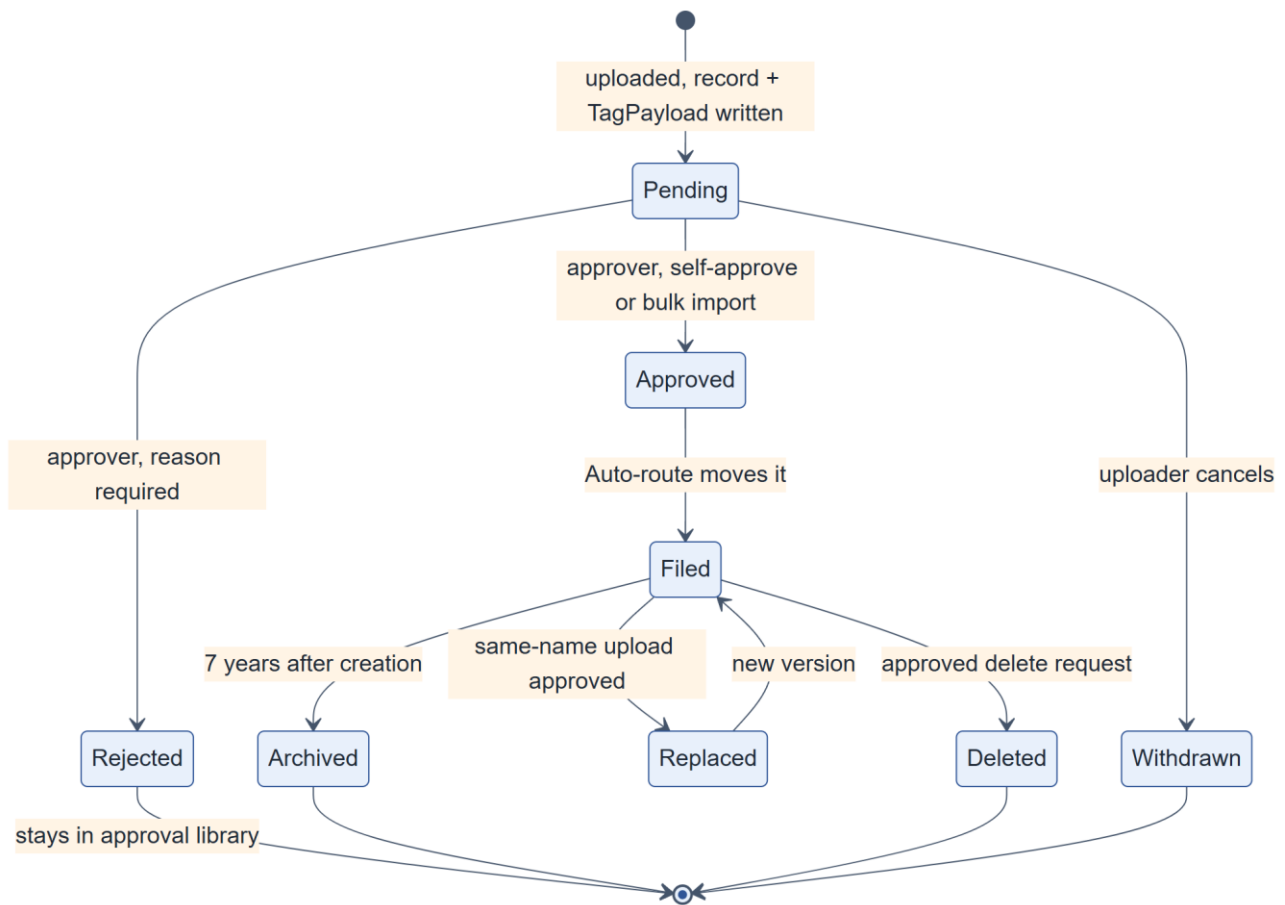


Figure 7. Document lifecycle

Tagging (CRS Submissions.TagStatus)

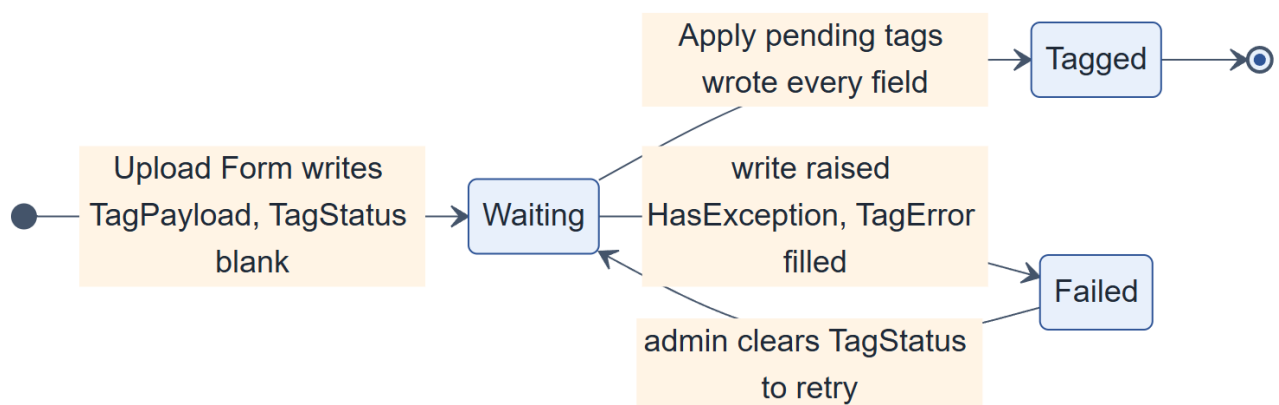


Figure 8. Tagging status

Request (CRS Requests.Status)

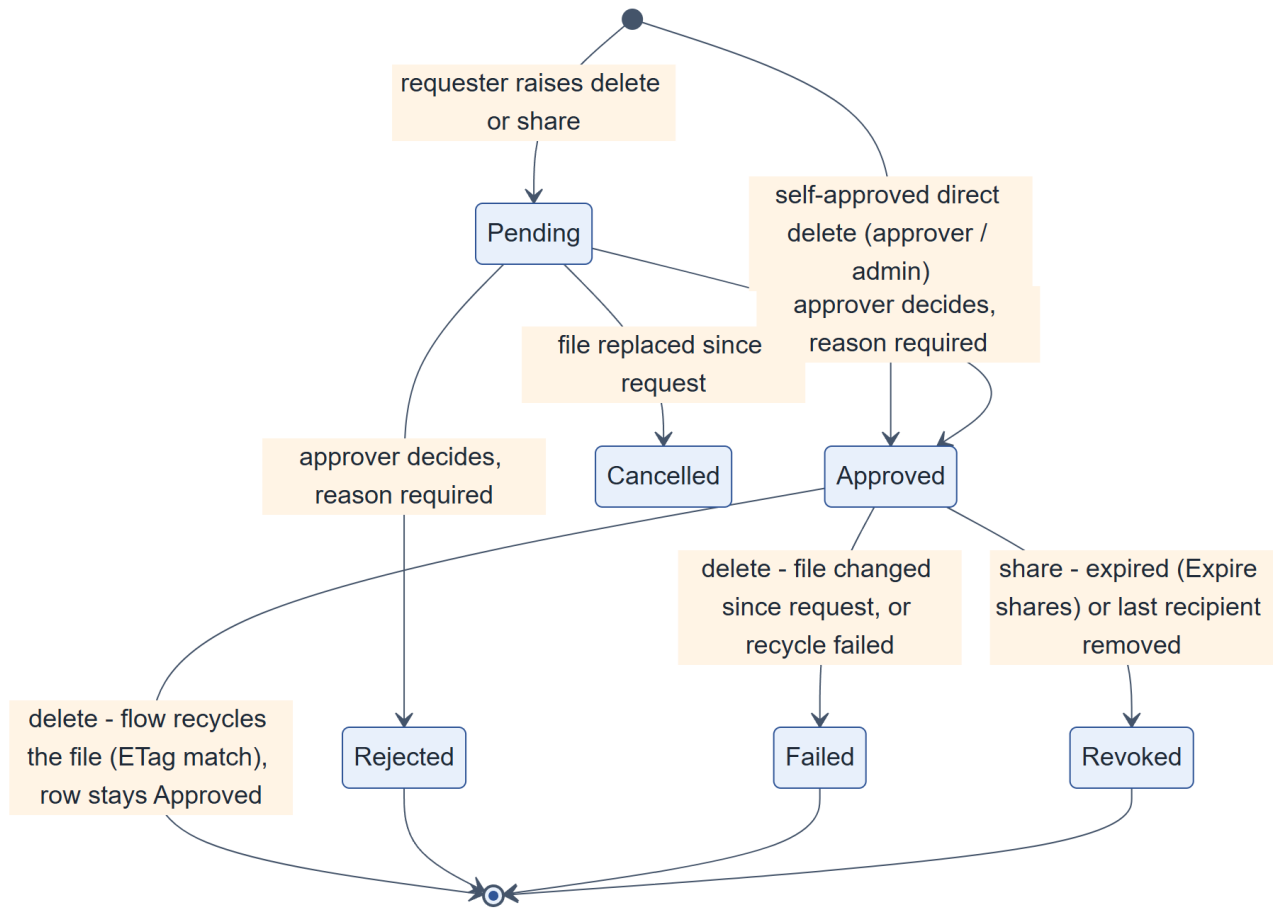


Figure 9. Request lifecycle

Submission record state shown on My Submissions

Table 28. Record states (precedence: live > withdrawn > cancelled > archived > deleted/unknown)

State	Meaning	Shown as
live	The file was found in a library (status read live).	Pending / Approved / Rejected
withdrawn	WithdrawnAt set: the uploader cancelled a pending file.	Cancelled
cancelled	ReplacedAt set: a newer upload replaced it.	Replaced
archived	ArchivedAt set, or found in an archive library.	Archived
deleted	Not found, and the library read was complete.	Deleted
settling	Just uploaded; the join is not expected yet.	Pending
unknown	A library read failed; nothing is concluded.	Unknown

4.10 Audit events

CRS Audit Log.EventType is free text, but every value must be one of the codes below (EVENT in src/shared/auditLog.ts). A new code must also be added to EVENT_LABEL and ALL_EVENT_TYPES, or the viewer cannot filter it.

Table 29. Audit event codes

Group	Event codes
Document	Uploaded, Approved, Rejected, Cancelled, Routed, Deleted, Archived, Replaced, UploadRefused

Group	Event codes
Sharing and access	ShareRevoked, AccessGranted, AccessRevoked
Requests	DeletionRequested, ShareRequested, RequestApproved, RequestRejected
Administration	ReconciliationRun, StructureChanged, MigrationRun, SegmentCreated, SegmentDeleted, SegmentRecoded, AbbreviationChanged, PolicyChanged, GroupMapChanged, GroupCreated, GroupDeleted, MembersChanged

- `writeAudit()` never throws and never blocks the action it records.
- Uploaded and Rejected rows come from the Audit approval activity flows; Approved and Routed from Auto-route; Deleted from the deletion flows; request events from Audit request activity. The Requests page writes no audit row on decide, to avoid duplicates.
- The viewer counts rows by walking ids; `ItemCount` is cached and wrong.

4.11 Volumes and limits

Table 30. Volumes and limits

Item	At capture	Limit or note
SharePoint groups	840	Site collection limit 10,000. About 6 groups per unit, 1 per department, 1 per segment.
CRS Group Map rows	1708	One row per group per role. Read with paging.
Units / departments / segments	132 / 40 / 3	From CRS Term Abbreviation and the mode rows
Role assignments on the web	845	Mostly Limited Access created by folder grants
Files per upload set	20	<code>MAX_FILES_PER_BATCH</code> ; Bulk Upload allows 50
List view threshold	5,000	Filter on indexed columns. The Audit Log is the list that will reach it.
<code>\$top</code> reads	—	A capped read cannot prove absence. Filter server-side or page with <code>nextLink / Id gt last</code> .
<code>/sitegroups</code>	—	Returns no <code>nextLink</code> : page by <code>Id gt <last></code> .

5. Security design

5.1 Identity and authentication

- Everyone signs in with their SD Guthrie Entra ID account. CRS has no login of its own.
- SPFx web parts call SharePoint with the signed-in user's token through `SPHttpClient`. The package requests no API permissions and uses no app-only identity, so the code can never do more than the user could do in SharePoint itself.
- Power Automate flows run under the connections of the service account `gdc@sdguthrie.com` (GDC Proxy - SharePoint, GDC Proxy - Outlook). When gdc's password changes the connections must be re-authenticated (Appendix D.2).
- A new group member must sign out and in again before an open session sees the new access.

5.2 Authorisation model

Access is plain SharePoint permissions. CRS decides which SharePoint group gets which permission level on which folder; SharePoint enforces it everywhere (libraries, search, direct links, the REST API). The code adds no second security layer; it only hides actions the user cannot perform.

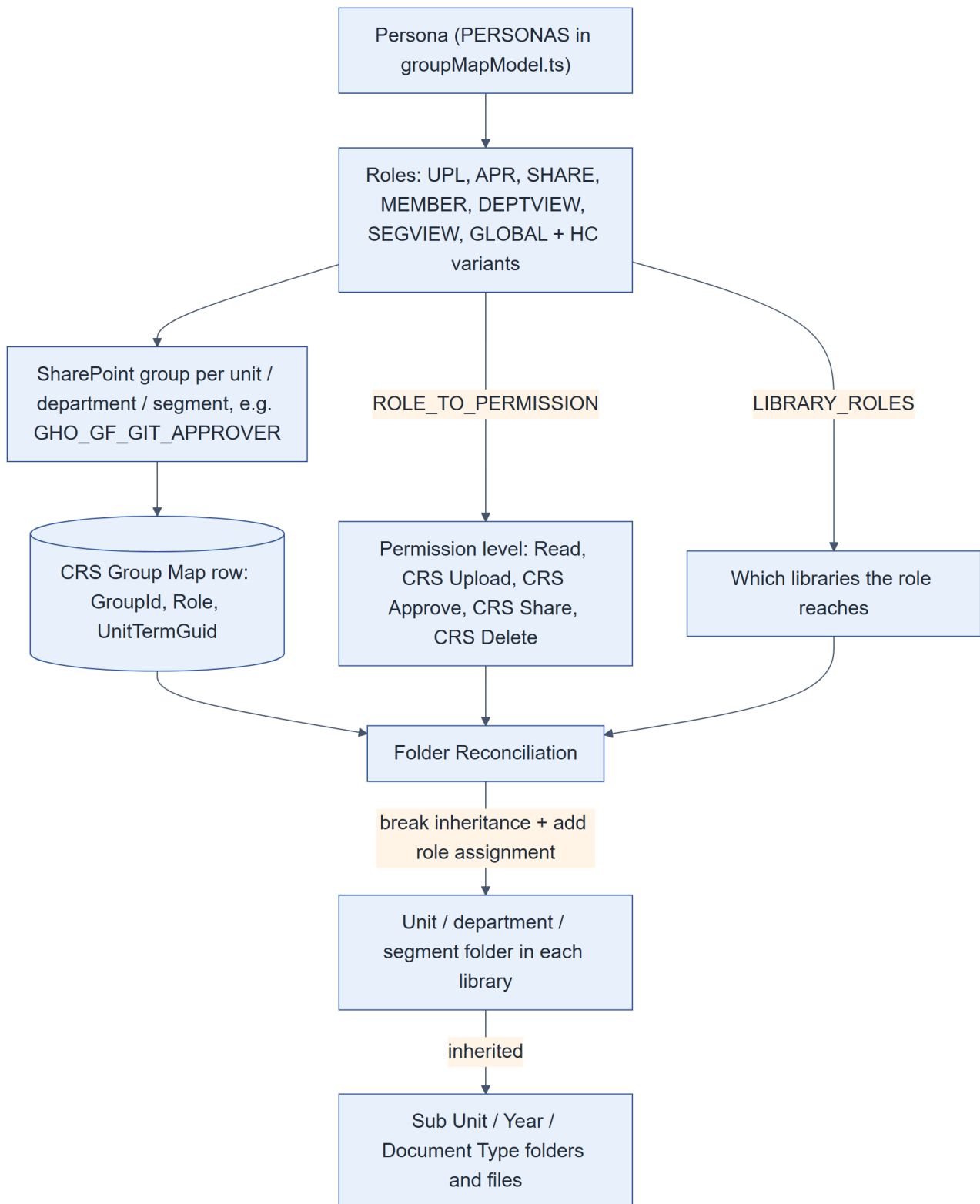


Figure 10. From persona to folder permission

1. An admin creates groups with Group Management. Each group is created for one persona at one term (unit, department or segment) and gets one CRS Group Map row per role in the persona.
2. Folder Reconciliation reads the Group Map and, for each row, grants the group the permission level for that role (ROLE_TO_PERMISSION) on the term's folder in each library the role reaches (LIBRARY_ROLES).
3. Permissions are broken at the segment, department and unit folders. Everything below a unit inherits from it.

- SharePoint adds Limited Access on the parents automatically, which is why the web and library roots carry hundreds of Limited Access entries.

5.3 Personas and roles

PERSONAS in `src/shared/groupMapModel.ts` is the only list of personas. A persona is matched by its exact role set.

Table 31. Personas (PERSONAS)

Persona (key)	Scope	Roles	Group name suffix
PIC (pic)	Unit	UPL	_UPLOADER
PIC HC (pic_hc)	Unit	UPLHC	_UPLOADER_HIGHLY_CONFIDENTIAL
Head of Unit (hou)	Unit	APR, SHARE, UPL	_APPROVER
Head of Unit HC (hou_hc)	Unit	APRHC, SHARE, UPLHC, SHAREHC	_APPROVER_HIGHLY_CONFIDENTIAL
SDG Employee (employee)	Unit	MEMBER	_VIEWER
SDG Employee HC (employee_hc)	Unit	MEMBER, MEMBERHC	_VIEWER_HIGHLY_CONFIDENTIAL
Head of Department (hod)	Department	DEPTVIEW, SHARE, SHAREHC	_HOD
C-Level segment (clevel_segment)	Segment	SEGVIEW	_C_LEVEL
C-Level global (clevel_global)	All segments	GLOBAL	C_LEVEL_GLOBAL

Group names are built from the folder codes, e.g. `GHO_GCA_EG_APPROVER` = segment GHO, department GCA, unit EG, Head of Unit. Live counts at capture:

Table 32. SharePoint groups by pattern (840 in total)

Group pattern	Count
Site default groups (Owners, Members, Visitors)	3
_SITE_MEMBERS	1
C_LEVEL_GLOBAL	1
_C_LEVEL	3
_HOD	40
_APPROVER	132
_APPROVER_HIGHLY_CONFIDENTIAL	132
_UPLOADER	132
_UPLOADER_HIGHLY_CONFIDENTIAL	132
_VIEWER_HIGHLY_CONFIDENTIAL	132
_VIEWER	132

Table 33. CRS Group Map rows by role

Role	Group Map rows
GLOBAL	1
SEGVIEW	3
DEPTVIEW	40
SHARE	304
SHAREHC	172
APR	132

Role	Group Map rows
UPL	264
APRHC	132
UPLHC	264
MEMBER	264
MEMBERHC	132

5.4 Roles, permission levels and libraries

The level each role gets in each library (`permissionForRole` in `FolderManager.tsx`). On the approved libraries the upload and approve roles are reduced to Read; on the archives every role is Read.

Table 34. Role × library permission matrix

Role	Approval	Documents	HC approval	HC documents	Archive	HC archive
UPL	CRS Upload	Read	—	—	—	—
APR	CRS Approve	Read	—	—	—	—
UPLHC	CRS Upload	Read	CRS Upload	Read	—	—
APRHC	CRS Approve	Read	CRS Approve	Read	—	—
SHARE	—	CRS Share	—	—	—	—
SHAREHC	—	—	—	CRS Share	—	—
MEMBER	—	Read	—	—	—	—
MEMBERHC	—	—	—	Read	—	—
DEPTVIEW	—	Read	—	Read	—	—
SEGVIEW	—	Read	—	Read	Read	Read
GLOBAL	—	Read	—	Read	Read	Read

- The grant is placed on the folder of the Group Map row's term: a unit folder for unit personas, the department folder for Head of Department (with `recon_departmentFanOut` it also reaches every unit below), the segment folder for SEGVIEW, and every segment for GLOBAL.
- System administrators need no row: `Guthrie Document Centre Owners` holds Full Control on the web and on every library.
- The delete roles (DEL, DELS, DELHC, DELSHC → CRS Delete) are no longer in any persona. Every delete is done by the service account (5.7).
- Pending and rejected drafts in the approval libraries are visible only to their author and approvers (Draft Item Security = author + approvers).

5.5 Permission levels

Table 35. Permission levels on the site (live)

Level	Kind	Purpose
Full Control	Built-in	Has full control.
Design	Built-in	Can view, add, update, delete, approve, and customize.
Edit	Built-in	Can add, edit and delete lists; can view, add, update and delete list items and documents.
Contribute	Built-in	Can view, add, update, and delete list items and documents.
Read	Built-in	Can view pages and list items and download documents.

Level	Kind	Purpose
CRS Upload	Custom	Uploaders. View and add items and folders; no Edit Items and no Delete Items (metadata is written by gdc).
CRS Approve	Custom	Approvers. View, Edit Items and Approve Items, without Add Items. Approve Items needs Edit Items in SharePoint.
CRS Delete	Custom	Read plus Delete Items. No persona holds a delete role since 17 Sep 2026; kept for history.
CRS Share	Custom	Sharers. Read plus Manage Permissions on their unit folder, used by the Requests page to grant access.
CRS Request	Custom	Granted on CRS Requests and CRS Submissions to every role group, so users can raise and track requests and records.

The exact base permissions of the custom levels are set on the site (Site settings › Site permissions › Permission levels). Check them there before changing a role mapping; the descriptions above give their design intent.

5.6 Where permissions are applied (live)

Table 36. Library roots

Library	Role assignments	Explicit grants other than Limited Access
Approval for Document	531	Guthrie Document Centre Owners (Full Control)
Restricted & Confidential Document	842	Guthrie Document Centre Owners (Full Control)
Approval for Highly Confidential Document	267	Guthrie Document Centre Owners (Full Control)
Highly Confidential Document	443	Guthrie Document Centre Owners (Full Control)
Archive Restricted & Confidential Document	8	Guthrie Document Centre Owners (Full Control)
Archive Highly Confidential Document	7	Guthrie Document Centre Owners (Full Control)

Table 37. Lists with unique permissions

List	Role assignments	Levels granted
CRS Requests	571	Full Control × 1, Read × 1, Edit × 1, CRS Request × 568
CRS Submissions	571	Full Control × 1, Read × 1, Edit × 1, CRS Request × 568
Site Pages	841	Full Control × 1, Limited Access × 839, Read × 2, Edit × 1
Site Assets	5	Full Control × 1, Limited Access × 3, Read × 2, Edit × 1

Table 38. Site-level group grants other than Limited Access

Group on the web	Levels
Guthrie Document Centre Owners	Full Control, Limited Access, Web-Only Limited Access
Guthrie Document Centre Visitors	Read
Guthrie Document Centre Members	Edit
GDC_SITE_MEMBERS	Read, Limited Access

- GDC_SITE_MEMBERS is the site-entry group: Read on Site Pages and Site Assets so every CRS user can open the pages. Folder grants alone only reach a folder by direct link; the site root would still deny.
- Guthrie Document Centre Members holds Edit on the web and on the request lists, and Visitors holds Read. Keep their membership to administrators: anyone added there bypasses the folder model.
- Admin pages (file names matching `folder|group.?(manag|config|setting|mapping|admin|access|audit)` are locked to Owners by reconciliation. Uploader pages (Upload Form, Bulk Upload, My Submissions) are granted to the UPL, UPLHC, APR and APRHC groups (`pageAccessPolicy.ts`, `pageGrants.ts`).

5.7 Service account and proxy actions

Table 39. Proxy actions

Action	Why the user cannot do it	Who does it
Write metadata on an upload	Uploaders hold CRS Upload, which has no Edit Items	CRS — Apply pending tags, from TagPayload
Move an approved file to Documents	Users have Read on the approved library	Auto-route flows
Delete a file	Nobody holds Delete Items	CRS — Execute approved deletion, from an Approved request row
Approve a bulk import	Historical files need no human approval	CRS — Auto-approve bulk imports
Approve new folders	A pending folder is invisible to other uploaders	CRS — Approve new folders
Revoke an expired share	Time-based	CRS — Expire shares
Archive after 7 years	Users have no rights on the archives	Archive flows
Send every email	One sender identity	All email flows, as gdc

gdc must stay in the Owners group. Removing it on 23 Sep 2026 stopped tagging, notifications and routing at once. Group Management hides gdc from the administrators list so it cannot be removed by accident. `crs@sdguthrie.com` is an ordinary account.

5.8 Highly Confidential separation

- HC documents live in their own approval, approved and archive libraries. Ordinary uploaders hold no permission there.
- The upload form offers the HC confidentiality level only if a live write probe on the HC unit folder succeeds (`useHcClearance`); group membership alone is not enough. The HC unit folder is resolved, never created by the form.
- Without the `hcConfidentialityLevel` config row every uploader would see the HC option.
- HC writes fail closed: they never fall back to the normal library.
- Only Head of Unit HC (APRHC) approves HC documents. Plain approvers receive only normal-library notifications.
- Every HC flow is a copy of the normal one pointed at the HC libraries (8.4). A missed library reference acts on the wrong file, because item ids are per list.

5.9 Security rules for developers

- Never add role checks in Search: SharePoint security trimming is the control.

- A failed permission read is unknown, never "no access" and never "access". Destructive and security decisions fail closed; form conveniences fail open.
- Reconciliation only adds grants, and fails closed if it cannot read the permission levels or the Owners group id.
- Deleting a CRS Group Map row revokes nothing. To take a role away: export members, delete the group, re-create it, re-add members, reconcile.
- SharePoint's native Approve/Reject and Share skip every CRS guard and are hidden by the hideAppBar extension. Keep it registered.
- Never hardcode a list or library title; a wrong title fails silently.

6. Component design

One section per component: what it does, what it reads and writes, how it works, and the rules that are deliberate and must not change without review. The large files are best searched by function name rather than read whole.

6.1 Upload Form

Item	Detail
Purpose	Main way documents enter CRS. The uploader picks a destination, fills one batch (set) of up to 20 files, and presses Upload.
Page / entry	Upload-Form.aspx (web part form)
Users	UPL, UPLHC, APR, APRHC groups; admins
Reads	CRS Config (mode rows, settings), term store, CRS Term Abbreviation, CRS Folder Map, CRS Group Map, approval and approved libraries (name clash)
Writes	Approval library (folders, files), CRS Submissions (record + TagPayload), CRS Requests (proxy delete when replacing a pending draft), CRS Audit Log
Main files	src/webparts/form/components/Form.tsx (6.7k lines), formModel.ts, uploadBatches.ts, dmsFolderMap.ts, spSubmissionRecords.ts, clashProxyDeletion.ts, hcClearance.ts

Processing

- Segment › tiers › Year › Document Type come from the mode row's Levels and the term store. A path is offered only if the leaf has a Folder Map row and an AddListItems probe on it succeeds (admins see every path, labelled when not fully set up).
- Each file gets Project, Vendor, Document Name, Date, Confidentiality, Legally Privileged (when required) and Keyword. File name = [Project] - [Vendor] - [Document Name] - [Date] (composeUploadBase).
- On Upload: re-check the chain (stale tab) and uploadsPaused; create missing below-Unit folders; upload each file; write a CRS Submissions row with TagPayload. The tagging flow writes the metadata as gdc (7.1).
- Name clash: a Yes/No dialog. Yes replaces: a pending draft is recycled by proxy (self-approved request row); an approved file is replaced by Auto-route only once the new one is approved.
- Self-approve (autoApproveOwnUpload = yes): for a Head of Unit, after TagStatus = Tagged, writes ApprovedBy in its own MERGE, then sets the moderation status, provided a live ApproveItems probe passes.
- A successful batch is removed; a failed one stays, so pressing Upload again is a safe retry.

Rules that must not change without review

- Only one batch is editable at a time; its destination is snapshotted at save (leaf term id), never read live at upload.
- Max 20 files per batch (MAX_FILES_PER_BATCH).
- A choice with a single option is greyed out for non-admins (segment and permissioned levels, isLevelLocked).
- The form never calls validateUpdateListItem itself; tagging is by proxy.
- HC is offered only with clearance; the HC unit folder is never created by the form.
- Styles are a CSS template literal: a backtick inside a CSS comment ends it.

6.2 Bulk Upload

Item	Detail
Purpose	Upload of historical, already-approved documents. One destination, up to 50 files.
Page / entry	Bulk-Upload.aspx
Users	Uploaders and approvers
Reads	CRS Config, term store, CRS Group Map, approval and approved libraries (duplicates)
Writes	Approval library with <code>BulkImport = true</code> , CRS Submissions
Main files	<code>src/webparts/bulkUpload/components/BulkUpload.tsx</code> (4.8k lines), <code>uploadBatches.ts</code>

Processing

- Writes files into the approval library with `BulkImport = true` and the level values (`buildLevelFormValues`).
- CRS — Auto-approve bulk imports approves them; Auto-route files them. No approved email is sent.
- Duplicates are checked against both the approval and the approved library; the clash dialog is the same as the Upload Form's.

Rules that must not change without review

- `BulkImport` must stay visible on both approval libraries; a hidden column is missing from the flow trigger.
- Files are matched by the `File` object, never by index or name.
- HC writes go through `writeSideSegment()` and fail closed.
- My Submissions shows these as Approved By Guthrie Document Centre, on purpose.

6.3 Approval Document and the Approve or reject panel

Item	Detail
Purpose	The approver reviews one pending file (page) or many (command-bar panel) and approves or rejects.
Page / entry	<code>Approval-Documents.aspx ?itemId=N (&lib=hc for HC; &decision=approve reject pre-ticks only)</code> . Panel: <code>bulkApprove</code> command set in the approval libraries.
Users	APR, APRHC groups; admins
Reads	Approval libraries, approved libraries (clash), CRS Config, permission probes
Writes	Approval library: <code>ApprovedBy</code> , <code>ApprovalComment</code> , moderation status and comment
Main files	<code>src/webparts/approvalDocument/components/ApprovalDocument.tsx</code> , <code>src/shared/approvalGuards.ts</code> , <code>approvalDestination.ts</code> , <code>src/extensions/bulkApprove/components/BulkApprovePanel.tsx</code>

Processing

- The approval panel shows only if the `ApproveItems` probe on the file says granted.
- Shared guards (`approvalGuards.ts`) check the decision is allowed: the destination unit is found by counting permissioned tiers from the top; `documentsFileClash` fails closed.
- Approve: MERGE `ApprovedBy` and `ApprovalComment` in one request, then set the moderation status in a second request. Reject: a reason is required.
- Prev / Next walk the pending queue (`FSObjType eq 0`, no folders). Auto-route does the rest (7.2).
- The panel handles a mixed selection per file and names each skipped file with its reason.

Rules that must not change without review

- Never set `OData__ModerationStatus` in the same MERGE as another field (HTTP 500).
- The panel and the page use the same guards; never copy them inline.
- The HC library's Name column formatting must carry `&lib=hc` (item ids are per list).
- React hooks stay above the early returns (the file returns early three times).

6.4 My Submissions and Document-Viewer

Item	Detail
Purpose	An uploader's own files across all six libraries, with history from CRS Submissions, and the place to raise delete and share requests.
Page / entry	My-Submissions.aspx; Document-Viewer.aspx ?file=<UniqueId> (same web part, viewer-only mode)
Users	Uploaders, approvers, admins
Reads	Six libraries (<code>AuthorId eq me</code>), CRS Submissions, CRS Requests, CRS Group Map, CRS Config, item permissions
Writes	CRS Requests (delete and share requests, self-approved direct deletes), CRS Submissions (withdrawn)
Main files	<code>src/webparts/mySubmissions/components/MySubmissions.tsx</code> (5.8k lines), <code>mySubmissions.ts</code> , <code>submissionRecords.ts</code> , <code>requests.ts</code>

Processing

- Tabs: All · Permission · Submissions · Pending · Approved · Rejected · Archive (a dropdown under 768 px, buttons from 768 px).
- Records are joined on the stamped `SubmissionFileId`, never `Uniqueld`. Status is always read live from the libraries; the record adds history (record state precedence in 4.9).
- Rights for an action = the item's live permissions (`probeFileRights`) or the Group Map; either can say yes.
- A direct delete is a self-approved CRS Requests row; the deletion flow recycles the file.

Rules that must not change without review

- A row is shown as deleted only if the library read was complete.
- On Document-Viewer, direct Delete is shown to system admins only, and an approved file shows no Approved badge or Approved-by box (client decision, 28 Sep 2026).
- Reads use `$select` fallback ladders; every read is cache-busted.
- The deep link must keep `?file=` in the URL (a refresh would go blank).

6.5 Requests

Item	Detail
Purpose	Approvers decide delete and share requests; the page also lists documents with shared access.
Page / entry	<code>CRS-Request.aspx</code> (<code>?request=<Id></code> opens one request; <code>?file=<UniqueId></code> gives direct actions from a library row)
Users	Approvers (their units), Heads of Department (department shares), admins (all), requesters (their own)
Reads	CRS Requests, CRS Submissions, CRS Config, CRS Group Map, file ACLs
Writes	CRS Requests (decision), file permissions (<code>SP.Web.ShareObject</code>), the list and its columns if missing
Main files	<code>src/webparts/requests/components/Requests.tsx</code> (4.4k lines), <code>requests.ts</code> , <code>directFileActions.ts</code> , <code>deletionGuard.ts</code>

Processing

- Three accordions: Delete Requests · Share Requests · Documents with Shared Access. Closed on landing, one open at a time, pending count in each header.
- `decide()` re-reads the row and refuses if it is no longer Pending. A reason is required to approve and to reject.
- Delete: writes Approved and the resolved `ItemUniqueId`; the deletion flow recycles the file if its ETag still matches `TargetETag`.
- Share: `SP.Web.ShareObject` with `sendEmail: false`; the flow sends the custom invite.
- If the request's submission record has `ReplacedAt`, the request cancels instead of executing.

Rules that must not change without review

- Who sees what is `inScope()` in `requests.ts` and nowhere else.
- No audit row is written on `decide` (the flow writes it).
- Statuses are parsed through `STATUS_SET`.
- Shared-access state is read live (live / revoked / unknown); Limited Access does not count as access.

6.6 Search

Item	Detail
Purpose	Home-page search: free text plus filters (Document Type, Year, Confidentiality, Segment, tiers).
Page / entry	Home.aspx (GDC Search web part)
Users	Everyone
Reads	SharePoint Search (approved and archive libraries), a 24-hour REST top-up of those libraries, term store, CRS Config, CRS Group Map
Writes	Nothing
Main files	<code>src/webparts/documentSearch/components/DocumentSearch.tsx</code> , <code>documentSearch.ts</code> , <code>throttleRetry.ts</code>

Processing

- KQL query over Documents, HC Documents, Archive and HC Archive through `/_api/search/query`.
- A direct REST read of the last 24 hours adds files not yet crawled (deduplicated on `Uniqueld`).
- Document Type, Year and Confidentiality are narrowed client-side by term GUID.

Rules that must not change without review

- Approval libraries are not searched (client decision).
- Security is SharePoint's trimming; never add role checks here.
- Search calls send `odata-version: ""` or they fail with 500.
- Stale results: re-index the library, and check the reindex flow.

6.7 Folder Administration and Folder Reconciliation

Item	Detail
Purpose	Guided admin flows (add segment, add department/unit, change structure, rename/re-code, retire, run reconciliation) and the reconciliation engine.
Page / entry	Folder-Administrator.aspx (admins only)

Item	Detail
Users	System administrators
Reads	CRS Config, term store, CRS Term Abbreviation, CRS Group Map, CRS Folder Map, permission levels, groups, folders
Writes	Folders in all six libraries, folder permissions, CRS Folder Map, CRS Term Abbreviation, page and request-list permissions, CRS Audit Log
Main files	FolderAdmin.tsx, FolderManager.tsx (9k lines; runReconciliation, ROLE_TO_PERMISSION, LIBRARY_ROLES, permissionForRole), AbbreviationManager.tsx, src/shared/folderFlows.ts

Processing

- Next is gated only by `blocksNext` in `folderFlows.ts`; the step rail is forward-locked.
- Reconciliation (7.8) builds folders, writes Folder Map rows, breaks inheritance on permissioned folders, grants from the Group Map, locks admin pages and grants the request lists.
- Abbreviations are unique among siblings and checked while typing. Saving writes rows only; reconciliation renames the folders.

Rules that must not change without review

- Reconciliation only adds permissions, and fails closed if it cannot read the permission levels or the Owners group.
- A scoped run skips the orphan and prune passes.
- Values read inside `runReconciliation` must be resolved inside the run (stale closures).
- `FolderManager.tsx` is the most site-tested code: change it narrowly.

6.8 Folder Structure and Migration

Item	Detail
Purpose	Changes a segment's folder levels, migrates existing files, and creates, retires or re-codes segments.
Page / entry	Inside the Folder Administration guided flows
Users	System administrators
Reads	CRS Config mode rows, term store, libraries
Writes	CRS Config Levels / PendingLevels and uploadsPaused, library columns (new levels), folders and files (migration), mode and Group Map rows (segments)
Main files	src/webparts/userAccess/components/StructureManager.tsx, SubtreeMigrator.tsx, SegmentCreator.tsx, src/shared/folderChain.ts, subtreeMigration.ts

Processing

- Saving on a segment in use stages `PendingLevels`; saving an unchanged chain clears it.
- Migration needs uploads paused (re-read, fails closed). A value is chosen per library per unit; files move, folders are created, clashes get a rename.
- The new chain goes live only after a fresh scan finds nothing left to move.
- Retire removes the mode row, Group Map rows and abbreviation rows, and the folders if ticked. Re-code works on empty segments only.

Rules that must not change without review

- Exactly two permissioned levels per segment; Year and Document Type are fixed.

- A per-unit level sits only directly under Unit, one per segment; a nested term set is refused for a shared level.
- The migrator walks leaf folders only; a file on a non-leaf folder is invisible to it.

6.9 Group Management

Item	Detail
Purpose	Creates the per-unit groups and their Group Map rows in bulk, manages members, answers "what can this person reach", and shows the system administrators.
Page / entry	Group-Management.aspx
Users	System administrators
Reads	Site groups and members, CRS Group Map, CRS Term Abbreviation, CRS Folder Map, term store
Writes	Site groups (create, delete, members), CRS Group Map, Owners membership (and Site Collection Admin)
Main files	GroupManager.tsx, BulkGroupProvisioner.tsx, GroupMembersEditor.tsx, src/shared/groupMapModel.ts, bulkGroups.ts, spGroups.ts

Processing

- Bulk provisioning builds every group a segment needs (one per persona per unit, _HOD per department, _C_LEVEL per segment) and loops until a pass writes nothing (max 10 passes).
- Groups are matched by term GUID after the name, so a renamed abbreviation does not create duplicates.
- Members are read with keyset paging (Id gt last).
- Reconciliation must run afterwards: creating a group grants nothing by itself.

Rules that must not change without review

- PERSONAS is the only role list; a persona is matched by its exact role set.
- Deleting a group removes its access at once; deleting a Group Map row does not.
- A persona change affects new groups only; existing groups must be deleted and re-created.
- gdc is hidden from the administrators card. Removing an admin never demotes yourself or the last admin.

6.10 Settings, Configuration and Access pages

Item	Detail
Purpose	GDC Settings is a directory of admin pages; GDC Configuration edits allowed file types; Page Access shows page grants; Approval Library Access, Site Access and Folder Access are signposts kept for old links.
Page / entry	CRS-Settings.aspx, CRS-Configuration.aspx, Page-Access.aspx, Approval-Library-Access.aspx, Site-Access.aspx, Folder-Access.aspx
Users	System administrators
Reads	Site Pages, CRS Config, page permissions
Writes	CRS Config allowedExtensions row and the AllowedFileTypes choices (Configuration only)
Main files	CrsSettings.tsx, adminPages.ts, backToSettings.tsx, FileTypeSettings.tsx, allowedFileTypes.ts, PageAccess.tsx, pageAccessPolicy.ts, pageGrants.ts

Processing

- Settings links are resolved from Site Pages at run time: resolved, ambiguous (shortest name wins) or missing (row disabled, names the page to create).
- Removing a file type unticks it first, then removes the choice. A Choices MERGE needs odata-version: "".

- Page Access writes nothing; reconciliation sets every page grant.

Rules that must not change without review

- Nothing ticked in AllowedFileTypes blocks every upload, on purpose. A missing column falls back to a default list with an admin warning.
- The Back to CRS Settings band is added in each web part's `render()`, never inside a shared component.

6.11 Audit Log

Item	Detail
Purpose	Filters, reads and exports CRS Audit Log.
Page / entry	CRS-Audit-Log.aspx
Users	System administrators
Reads	CRS Audit Log
Writes	Creates the list if missing
Main files	<code>src/webparts/auditLog/components/AuditLog.tsx</code> , <code>src/shared/auditLog.ts</code> , <code>spAuditLog.ts</code> , <code>displayName.ts</code>

Processing

- Filters: date, action, person, keyword. The Who column shows the person, or Guthrie Document Centre for automated steps (`resolveActorDisplay`).
- Totals are counted by walking ids. The CSV export keeps the stored actor.

Rules that must not change without review

- EventType is Text, never Choice; every event a flow writes must be in EVENT, EVENT_LABEL and ALL_EVENT_TYPES.
- `writeAudit` never throws and never blocks.

6.12 Extensions

Item	Detail
Purpose	<code>hideAppBar</code> : adds + New Folder and hides SharePoint's native Upload, Approve/Reject and Share in CRS libraries. <code>bulkApprove</code> : the Approve or reject panel.
Page / entry	Every CRS library
Users	Everyone (<code>hideAppBar</code>); approvers (<code>bulkApprove</code> , shown only when the permission probe says granted)
Reads	Library permissions, CRS Config
Writes	Folders (+ New Folder); moderation fields (panel, as 6.3)
Main files	<code>src/extensions/hideAppBar/HideAppBarApplicationCustomizer.ts</code> , <code>src/extensions/bulkApprove/BulkApproveCommandSet.ts</code> , <code>components/BulkApprovePanel.tsx</code> , <code>sharepoint/assets/elements.xml</code>

Processing

- `hideAppBar` works only on libraries whose title is in CRS_LIBRARIES; elsewhere it switches itself off.
- `bulkApprove` sets `cmd.visible` inside the probe's promise, so one click is enough.

Rules that must not change without review

- Both were registered by hand in `/_api/web/usercustomactions` on this tenant (Appendix A.5). Two registrations load an extension twice.
- Add a renamed library title to `CRS_LIBRARIES`.
- `uploadCommand` is legacy and not registered.

7. Process design

7.1 Upload and tagging

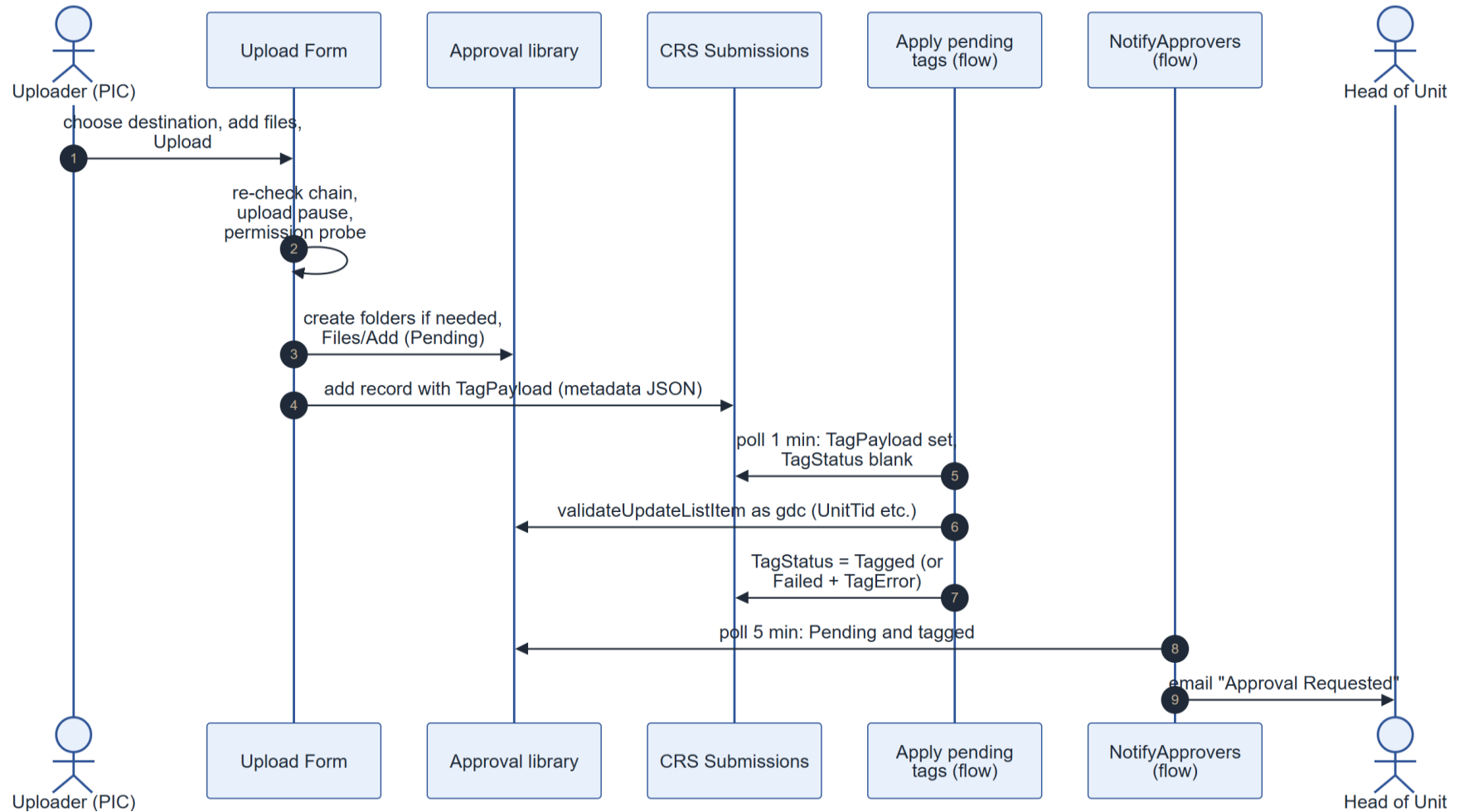


Figure 11. Upload, tagging and approver notification

- The file lands Pending in the approval library as the uploader; its metadata arrives up to a minute later when the tagging flow runs.
- If tagging fails, TagStatus = Failed and TagError says why. NotifyApprovers waits for UnitTid, so no approver email is sent until tagging succeeds.
- Self-approval waits for TagStatus = Tagged, then approves (6.1).

7.2 Approval and routing

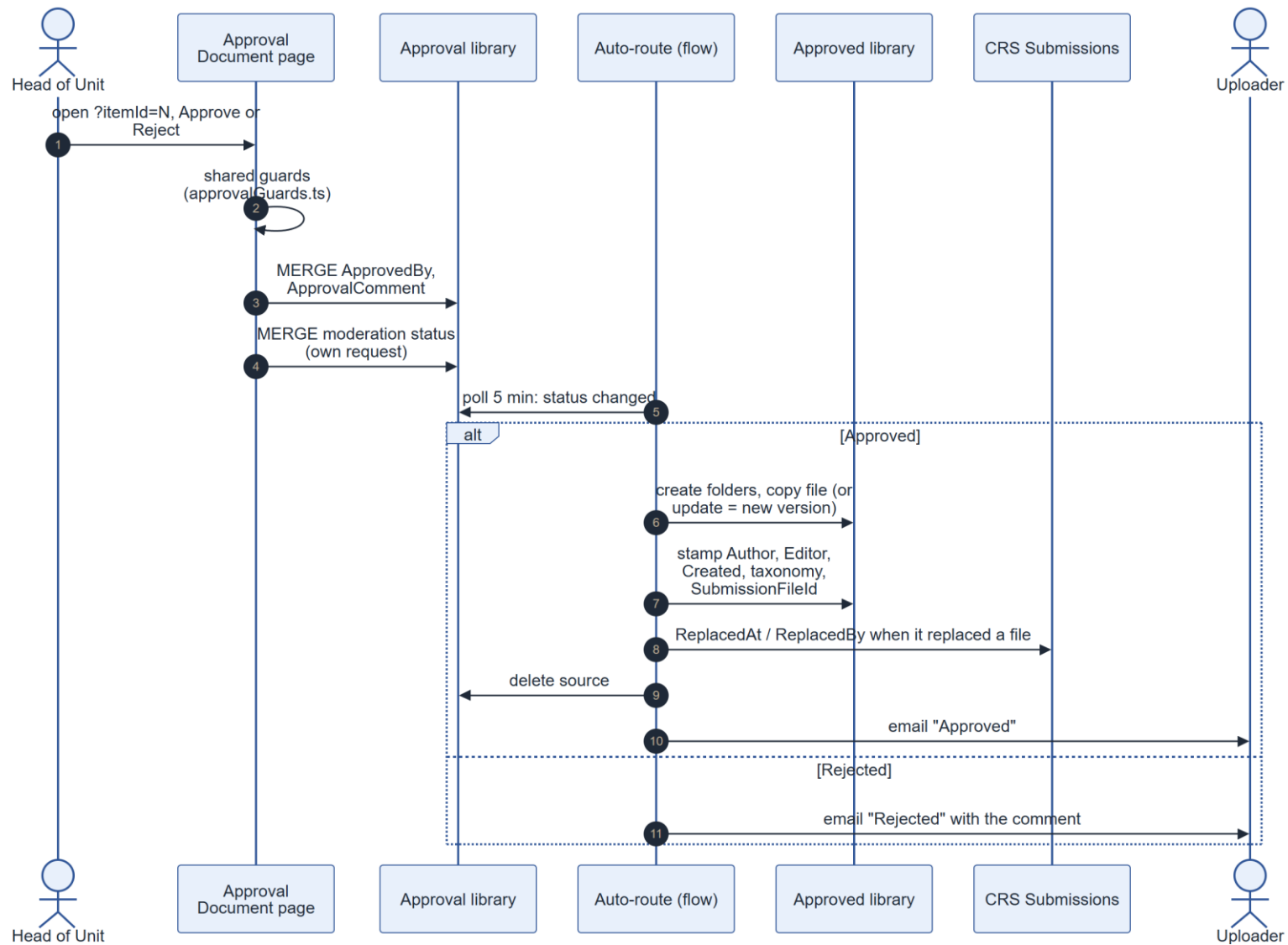


Figure 12. Approval and Auto-route

- Auto-route polls every 5 minutes, so an approved file appears in the approved library within about 5 minutes.
- If a same-named approved file exists, Auto-route updates it (a real new version) and stamps ReplacedAt / ReplacedBy on the old record.
- SharePoint's copy drops managed metadata, so Auto-route copies Document Type, Year and Confidentiality explicitly.
- The approved email is skipped for bulk imports, self-approvals and uploaders who are site owners.
- A rejected file stays in the approval library so the uploader can read the comment.

7.3 Bulk import

1. Bulk Upload writes files to the approval library with `BulkImport = true` and the level values.
2. CRS — Auto-approve bulk imports (polls every 5 min) sets the moderation status to Approved.
3. Auto-route files them as in 7.2, without the approved email.

7.4 Approver notifications and reminders

Live on CRS at capture: NotifyApprovers (+ HC) sends one email per file, and the reminder flows email each approver per due file every 3 days. Built and tested on the test site, ready for CRS: the bundled approver email and the bundled reminder.

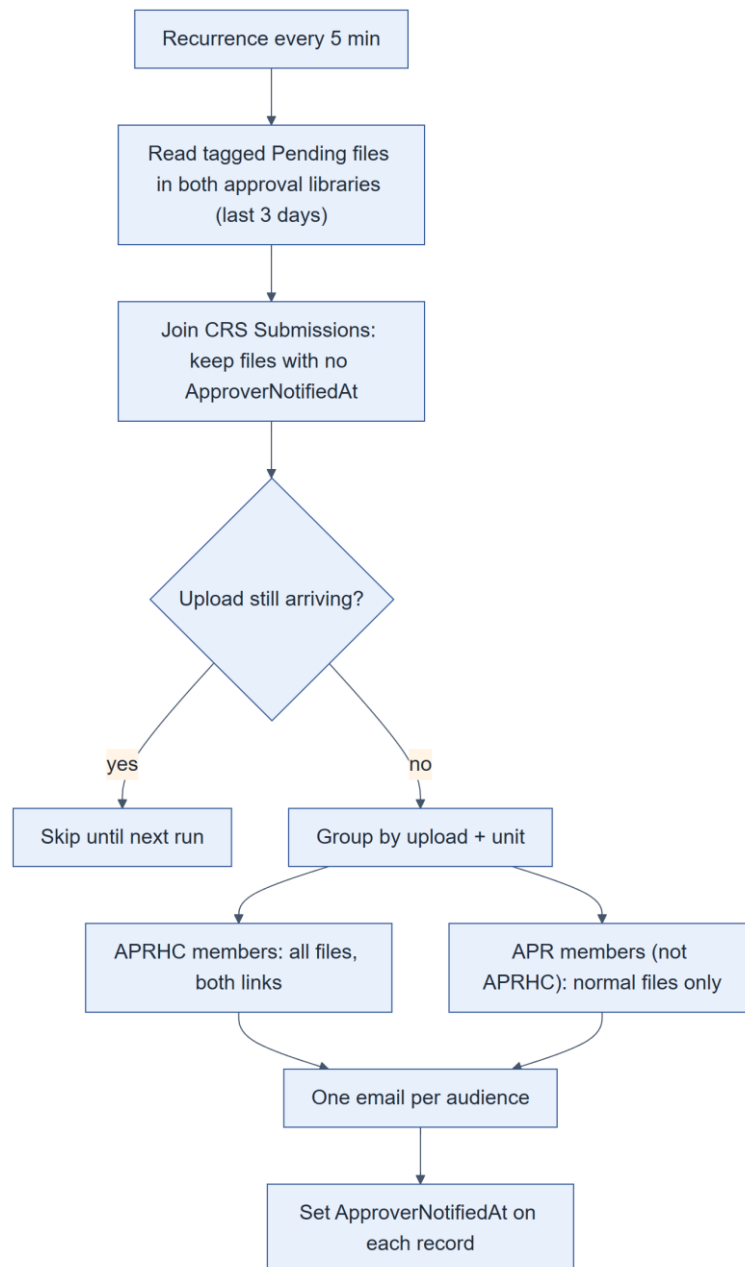


Figure 13. Bundled approver email (replaces the per-file email)

- Due rule for reminders: Pending, not a folder, not a bulk import, and NextReminderAt has passed (or is empty and the file is over 3 days old). After sending, NextReminderAt moves 3 days on.
- The bundled reminder reminds about every pending file of an upload once any of them is due, and resets all of them, so an upload never splits across reminders.
- Audiences: APRHC members get every file and both library links; plain APR members get only normal files. Reminders leave out the uploader, gdc and site admins; the first approval email includes site admins.

7.5 Delete request

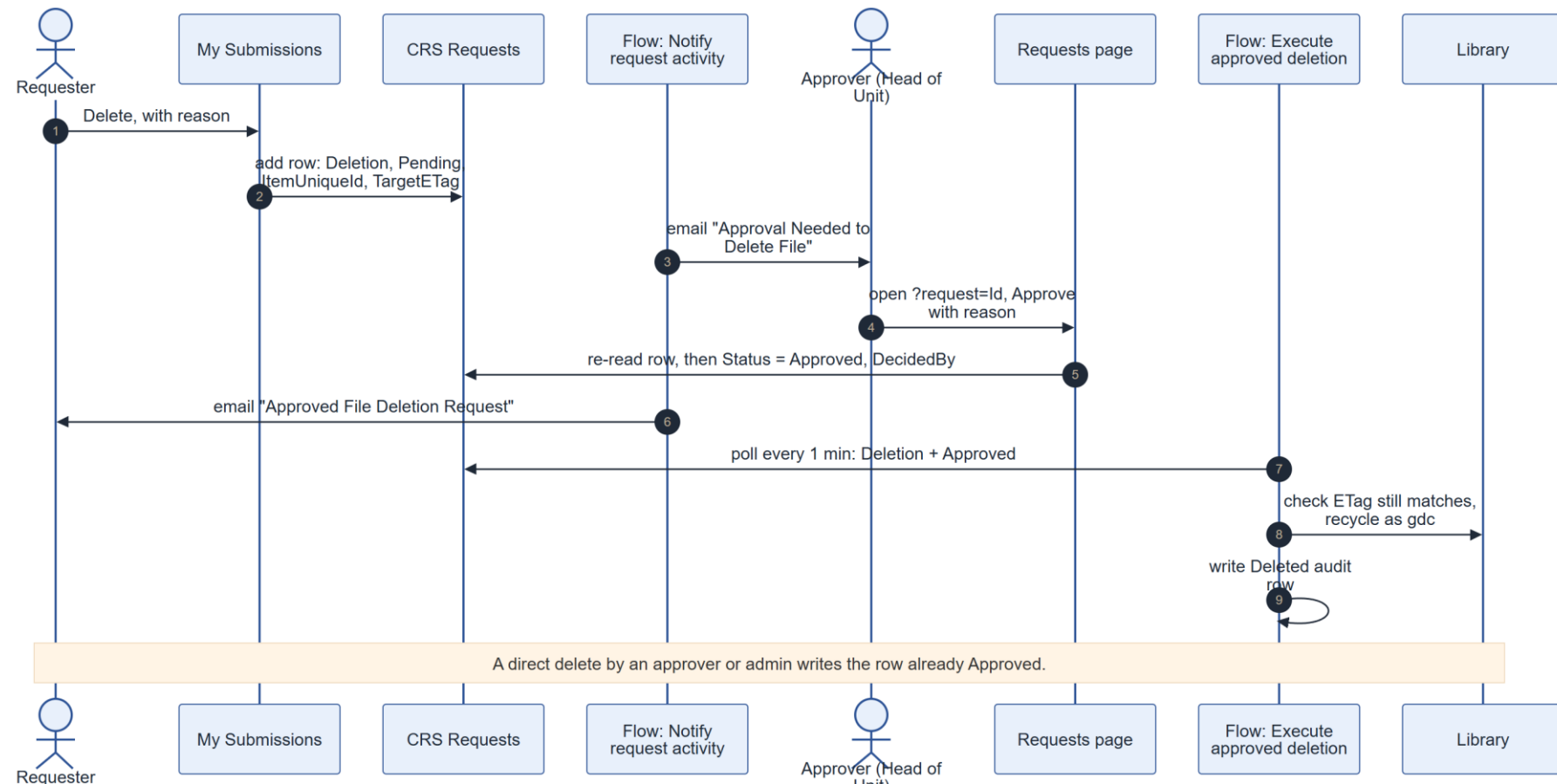


Figure 14. Delete request and proxy deletion

- TargetETag is stamped when the delete is decided. If the file changed since, the flow refuses and marks the request Failed with a note.
- When an uploader deletes their own pending draft (cancel), WithdrawnAt is set on the record and My Submissions shows Cancelled.

7.6 Share request and expiry

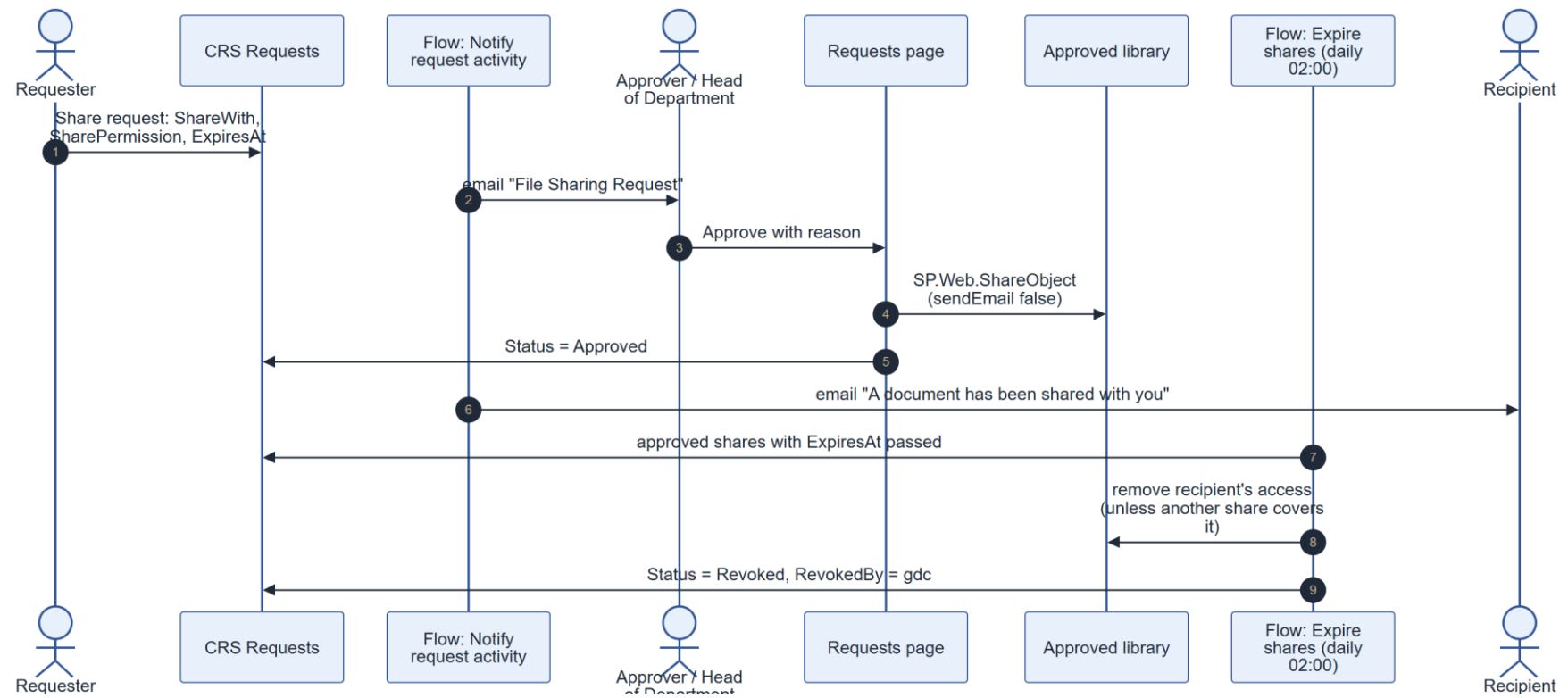


Figure 15. Share request, grant and expiry

- A blank `ExpiresAt` means the share is permanent and is never revoked by the flow.
- Revoked is written only when the last recipient's access goes; a partial revoke appends to `DecisionNote`.
- Outside addresses are allowed only when `allowExternalSharing` = yes; domains in `tenantDomains` count as internal. The tenant's mail-flow rule blocks email to outside addresses unless SD Guthrie IT adds an exception.

7.7 Seven-year archive

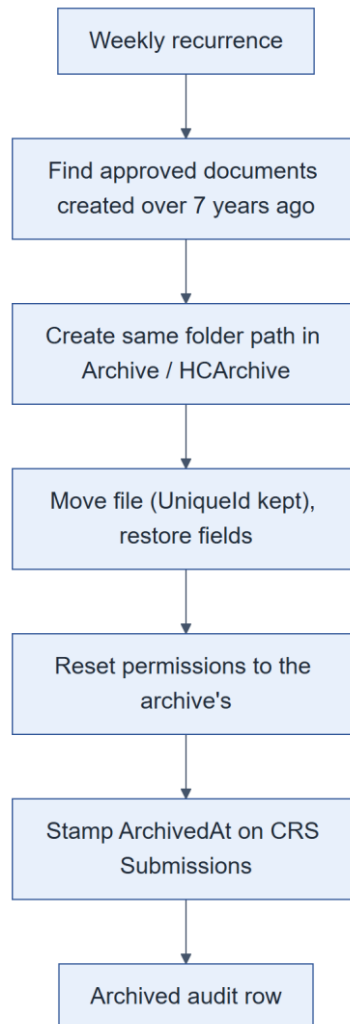


Figure 16. Archive after seven years (weekly, normal and HC flows)

7.8 Folder reconciliation

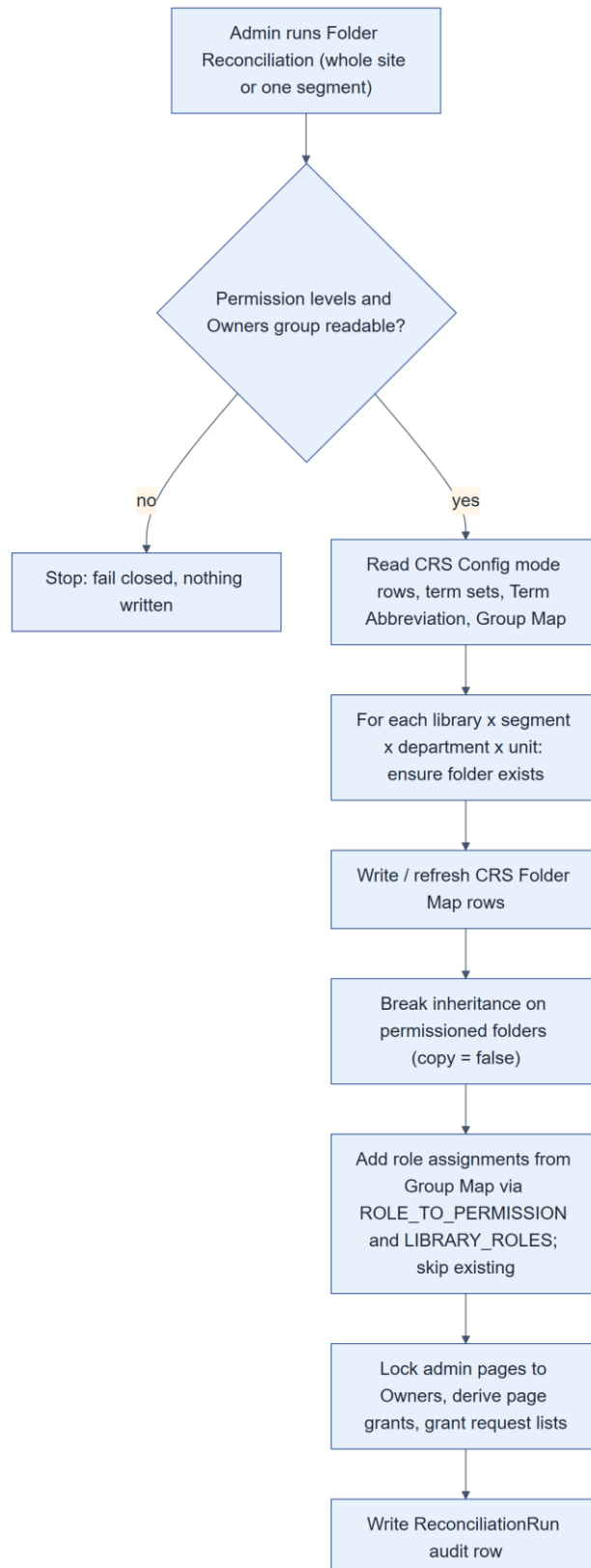


Figure 17. Folder Reconciliation

- Writes are paced (200 ms between writes, a cooldown every 300 writes) to avoid throttling. A full run can take tens of minutes on the live site.
- Grants that already exist are skipped, so a re-run is safe.

7.9 Adding a unit (end to end)

1. Add the department and unit terms in the site term store (rename existing terms; never re-create them).
2. Folder Administration › Add department/unit: set the abbreviations.
3. Group Management › Bulk Provisioning for the segment: creates the unit's groups and Group Map rows.
4. Add members to the groups.
5. Folder Administration › Run reconciliation (scoped to the segment): creates the folders in all six libraries and grants the groups.
6. Test with a non-admin uploader and approver.

8. Integration design: Power Automate

About half of CRS's behaviour lives in Power Automate, not in the code. The exports in PowerAutomateFlowsSDG/ (25 September 2026) are the source of truth: unzip one and read definition.json.

8.1 Environment and connections

Table 40. Power Automate environment

Item	Detail
Owner	gdc@sdguthrie.com ("Guthrie Document Centre"). Every flow runs as it.
Environment	SD Guthrie default environment, https://make.powerautomate.com . Sign in as gdc to see them under My flows.
Connections	GDC Proxy - SharePoint (every list and file step) and GDC Proxy - Outlook (every email). Both signed in with gdc's password.
Site	Every flow names https://sdguthrie.sharepoint.com/sites/CRS and CRS list GUIDs (8.6).
Run history	28 days per flow; each step's inputs and outputs are visible (Show raw outputs).
Trigger latency	SharePoint triggers poll: 1 minute (tagging, deletion) or 5 minutes (others). A recurrence flow catches up on its next run; a trigger flow that is off misses changes made while it was off.

8.2 Flow inventory

Table 41. Flows (25 live, 2 ready to deploy)

Flow	Trigger	Purpose
Auto-route approved Pending files to Documents Library (+ HC Auto-route)	Approval library item created/modified, polls 5 min, files only	On approve: create folders, copy or update the file, stamp metadata and ids, mark replaced records, delete the source, audit, email the uploader. On reject: email the uploader with the comment.
CRS — Apply pending tags	CRS Submissions created/modified, polls 1 min; TagPayload set and TagStatus empty	Find the file by ItemUniqueId, write TagPayload with validateUpdateListItem as gdc, set TagStatus / TagError.
CRS — Auto-approve bulk imports (+ HC auto-approve)	Approval library item modified, polls 5 min; BulkImport and Pending	Set the moderation status to Approved.
CRS — Approve new folders (+ HC folder approval)	Approval library item created, polls 5 min; folders only	Approve the folder so other uploaders can see and navigate it.
NotifyApprovers (+ HCNotifyApprovers)	Approval library item created/modified, polls 5 min; Pending, not bulk	When UnitTid is set and NextReminderAt empty: find the unit's APR / APRHC group in CRS Group Map, email its members except the uploader. Bcc gdc.
CRS — Reminding Approver to Approve (+ CRS — HC approval reminder)	Recurrence, 30 min	Email each approver of each due file (max 10), then move NextReminderAt 3 days on.
CRS — Notify request activity	CRS Requests created/modified, polls 5 min	New request: email the approver group. Decision: email the requester. Approved share: email the recipient.
CRS — Execute approved deletion	CRS Requests modified, polls 1 min; Deletion + Approved	Check the file's ETag against TargetETag, recycle it as gdc, write a Deleted audit row; mark Failed with a note if it cannot.
CRS — Expire shares	Recurrence, daily 02:00 (Singapore time)	Remove expired recipients' access (unless another approved share covers them), set Revoked.

Flow	Trigger	Purpose
CRS — Archive after seven years (+ HC)	Recurrence, weekly	Move approved documents older than 7 years to the archive, restore fields, reset permissions, stamp ArchivedAt, audit.
CRS — Nightly library reindex	Recurrence, every 6 hours	Toggle NoCrawl on the approved and archive libraries so search re-crawls them.
Audit — approval activity (+ HC)	Approval library item created/modified, polls 5 min	Write Uploaded and Rejected audit rows, once per file and event.
Audit — approval deletions, Documents deletions (+ HC: four flows)	Item deleted, polls 5 min	Read the recycle bin for who and when; write a Deleted row (skips files Auto-route just moved).
CRS — Audit request activity	CRS Requests modified, polls 5 min	Write request events to the audit log, once each.
CRS — Audit replacements	CRS Submissions modified, polls 5 min; ReplacedAt set	Write Replaced events.
CRS — Notify approvers (bundled) · test site only	Recurrence, 5 min	One email per upload per unit per audience; sets ApproverNotifiedAt. Replaces NotifyApprovers. Built by scripts/build-bundled-approver-flow.py.
CRS — Remind approvers (bundled) · test site only	Recurrence, 30 min, one run at a time	Bundled reminders; replaces both reminder flows. Built by scripts/build-bundled-reminder-flow.py.

8.3 Which flows listen to which list

Writing a field re-triggers every "created or modified" flow on that list. Check this table before adding a write.

Table 42. Flow triggers by list

List or library	Event	Flows
Approval for Document / HC	Created or modified	Auto-route, Auto-approve bulk imports, NotifyApprovers, Audit approval activity (and their HC twins)
Approval for Document / HC	Created (folders)	Approve new folders (+ HC)
Approval for Document / HC	Deleted	Audit approval deletions (+ HC)
Documents / HC Documents	Deleted	Audit Documents deletions (+ HC)
CRS Submissions	Created or modified	Apply pending tags, Audit replacements
CRS Requests	Created or modified	Notify request activity, Execute approved deletion, Audit request activity
(schedule)	Recurrence	Reminders (30 min), Expire shares (daily), Archive (weekly), Reindex (6 h), bundled flows (5 / 30 min)

8.4 Data contract between code and flows

Renaming, deleting or hiding one of these columns breaks a flow silently.

Table 43. Columns the flows depend on

Where	Column	Used by (R = reads, W = writes)
Approval libraries	BulkImport	R: NotifyApprovers, Auto-approve bulk imports (trigger), reminders, Auto-route (email skip). Must stay visible.
Approval libraries	UnitTid	R: NotifyApprovers, reminders, bundled flows (approver group lookup).
Approval libraries	NextReminderAt	W: reminder flows. R: NotifyApprovers, bundled approver email, bundled reminder.

Where	Column	Used by (R = reads, W = writes)
Approval libraries	ApprovedBy	R: Auto-route (approver name, self-approval check), Audit approval activity. Written by the app.
Approval libraries	OData__ModerationStatus, OData__ModerationComments	W: Auto-approve, Approve new folders. R: Auto-route (decision, rejection comment), NotifyApprovers.
Approval libraries	SubmissionId, BatchId, SubmissionFileId	R: Auto-route (copied to the approved file), bundled flows.
Approval libraries	Document_x0020_Type, Year, Confidentiality_x0020_Level	R: Auto-route (copied explicitly), emails.
Approved libraries	SubmissionFileId, Created	R: archive flows (7-year cut-off, ArchivedAt stamp), Auto-route (replacement).
CRS Submissions	TagPayload, TagStatus, TagError, ItemUniqueld	R/W: Apply pending tags.
CRS Submissions	ReplacedAt, ReplacedBy	W: Auto-route. R: Audit replacements.
CRS Submissions	ArchivedAt	W: archive flows.
CRS Submissions	ApproverNotifiedAt, SubmissionRef, UploadedAt	R/W: bundled approver email (column to be added on CRS).
CRS Requests	RequestType, Status, ItemUniqueld, RequestedBy, DecidedBy, ShareWith, ExpiresAt, TargetETag, UnitTermGuid, DecisionNote, RevokedBy	Notify request activity, Execute approved deletion, Expire shares, Audit request activity.
CRS Group Map	Role, UnitTermGuid, GroupId	R: NotifyApprovers, reminders, Notify request activity, bundled flows.
CRS Audit Log	Title, EventTime, EventType, Outcome, ActorName, ActorEmail, Source, LibraryName, ItemUniqueld, ItemName, ItemPath, Details, Segment, UnitPath	W: every audit flow, Auto-route, archive flows, Execute approved deletion.

8.5 Emails

All emails are sent as gdc through GDC Proxy - Outlook. Email to addresses outside SD Guthrie is blocked by the tenant's mail-flow rule; the bounce lands in gdc's inbox.

Table 44. Emails

Subject	Sent by	To	Bcc
Approval Requested: <file>	NotifyApprovers (+ HC)	Unit approvers (site admins included), not the uploader	gdc
REMINDER: Approval Required for <file>	Reminder flows	Each approver separately (max 10); not the uploader, site admins or gdc	—
Approved: <file> has been approved	Auto-route (+ HC)	Uploader (not for bulk imports, self-approvals or owner uploaders)	—
Rejected: <file> has been rejected	Auto-route (+ HC)	Uploader	—
Approval Needed to Delete File: <name>	Notify request activity	Unit approvers	gdc
File Sharing Request: <name>	Notify request activity	Unit approvers	gdc
Approved / Rejected File Deletion Request	Notify request activity	Requester (not if they decided it themselves)	—
Approved / Rejected File Sharing Request	Notify request activity	Requester (not if they decided it themselves)	—

Subject	Sent by	To	Bcc
A document has been shared with you: <name>	Notify request activity	The share recipients	—
Approval Requested: <file> / N documents	Bundled approver email (test site)	APRHC / APR audiences	gdc (client deciding)
REMINDER: Approval Required: <file> / N documents	Bundled reminder (test site)	Same audiences, minus site admins	As above

8.6 List GUIDs the flows use (CRS)

Table 45. CRS list GUIDs

List	GUID
Approval for Document	eeb1bb19-ec53-4c41-8dc9-ecf255979c9b
Approval for Highly Confidential Document	d935aa6d-dc48-4832-ba7e-eca485ecdff3
Restricted & Confidential Document	fb062dd-fcd0-4458-9161-2bfc19c917fa
Highly Confidential Document	122a4aa9-65fb-479a-90a7-3866d19f51b4
Archive Restricted & Confidential Document	647fd644-ab00-4358-9b3f-49031083fcf0
Archive Highly Confidential Document	fc301054-abf5-4e53-957a-6f8c58a410b8
CRS Submissions	dd1bc5bd-daa7-491a-9dc7-b81212c391ab
CRS Requests	1bcb1aa1-fbf1-4d1b-a6d2-88436824c297
CRS Audit Log	4ac219e5-b855-46a3-be4b-4e96bba546c5
CRS Group Map	5acf3f3e-d955-4a82-8a49-24db3a353514
CRS Config	22e4f613-1693-407c-9f94-318088c51775

Some flow actions name lists by title instead (e.g. `getbytitle('CRS Group Map')`); those keep working on another site with the same titles. Item ids are per list.

9. Interface design

9.1 User interface map

Table 46. Pages and deep links

Page	Who	Deep-link parameters
Home.aspx (Search)	Everyone	—
Upload-Form.aspx	UPL, UPLHC, APR, APRHC	—
Bulk-Upload.aspx	UPL, UPLHC, APR, APRHC	—
My-Submissions.aspx	UPL, UPLHC, APR, APRHC	—
Document-Viewer.aspx	Anyone with access to the file	?file=<UniqueId>
Approval-Document.aspx	Approvers	?itemId=<id>, &lib=hc for HC, &decision=approve reject (pre-tick only)
CRS-Request.aspx	Approvers, HoD, requesters, admins	?request=<Id>, ?file=<UniqueId> (strip braces)
CRS-Settings.aspx and admin pages	Owners only	—
Library views	By permission	Name column formatting links to Approval-Document / Document-Viewer

- Debug a page against local code:
?debug=true&noredir=true&debugManifestsFile=https://localhost:4321/temp/build/manifests.js.
- Every admin page shows a Back to CRS Settings band.
- Layouts are phone-first; status tabs become a dropdown under 768 px.

9.2 SharePoint APIs used

Table 47. REST calls per component

Component	Lists	Main REST calls
Upload Form	Config, Group Map, Folder Map, Term Abbreviation, Submissions, Audit Log	/_api/v2.1/termStore/sets/{id}/children; GetFolderByServerRelativeUrl, folders/AddUsingPath; Files/Add; /items; EffectiveBasePermissions; approve()
Bulk Upload	Config, Group Map	Term store v2.1; Files/Add; validateUpdateListItem; currentUser
Approval Document	Approval libraries, Config	/items; approve(); MERGE OData__ModerationStatus; file lookups
Approve or reject panel	Approval libraries, Config	MERGE ApprovedBy, then MERGE OData__ModerationStatus (two requests)
My Submissions / Document-Viewer	Six libraries, Submissions, Requests, Group Map, Config	/items with \$select ladders; currentUser
Requests	Requests, Config, Group Map, Audit Log	/items; list and column creation (/_api/web/lists, CreateFieldAsXml); role assignments; SP.Web.ShareObject
Search	Approved and archive libraries, Config, Group Map	/_api/search/query; term store v2.1; TaxonomyHiddenList
Folder Administration	Config, Group Map, Folder Map, Term Abbreviation, Submissions, Requests	Term store v2.1; folders/AddUsingPath; breakroleinheritance, roleassignments/addroleassignment; sitegroups; list and column creation

Component	Lists	Main REST calls
Group Management, access pages	Config, Group Map, Term Abbreviation, Folder Map	sitegroups (create, members, delete); ensureuser; role assignments; term store v2.1
GDC Settings / Configuration	Site Pages, Config	/items; field Choices MERGE
Audit Log	Audit Log	/items; list creation if missing

9.3 REST conventions

- One unknown column in \$select, or one unknown field in a write, fails the whole call. Optional columns are read in fallback steps.
- validateUpdateListItem returns HTTP 200 even when a field fails: check HasException. Dates go as M/D/YYYY there, ISO in /items and \$filter.
- Taxonomy values are Label|GUID. Compare terms by GUID.
- Yes/No in \$filter is 1/0, not true/false.
- Deep paths use the alias form GetFolderByServerRelativeUrl(@f)?@f='...'.
- CAML needs internal names (FSObjType, _ModerationStatus); REST uses FileSystemObjectType, OData__ModerationStatus.
- FieldValuesAsText double-encodes underscores in keys (e.g. Department_x005f_x0020_x005f_Type).
- Search and Choices MERGE calls need the header odata-version: "".
- A read capped with \$top cannot prove something is missing. Reads that decide an action are cache-busted.

10. Non-functional design

Table 48. Quality attributes

Quality	Design
Performance	Reads are filtered server-side on indexed columns; queues exclude folders; search uses the index plus a 24-hour top-up. Large admin runs are paced to avoid throttling.
Latency	Tagging up to 1 min; routing, approver email and audit up to 5 min; reminders every 30 min; archive weekly; search re-crawl every 6 h.
Scalability	About 6 groups per unit; 840 groups for 132 units, against a 10,000 limit. The 5,000-item list view threshold is handled by indexed filters and paging; the Audit Log is the list that will grow largest.
Reliability	Upload retries are safe (success removes, failure stays). Audit-writing flows are idempotent (once per file and event). The delete flow checks the ETag before acting.
Error handling	A failed read is unknown, never empty. Form gates fail open; destructive and security decisions fail closed. <code>validateUpdateListItem</code> results are checked for <code>HasException</code> . 429/503 are retried.
Auditability	Every upload, decision, routing, deletion, share, archive and admin change writes a CRS Audit Log row. Writing an audit row never blocks the action.
Security	Chapter 5: SharePoint permissions only, user-context REST, proxy actions as gdc, HC isolation.
Maintainability	Rules in pure tested modules (<code>src/shared/x.ts + x.test.ts</code>); configuration in lists; names resolved at run time; flows exported to Git.
Testability	<code>npm run heft test --clean</code> runs the unit tests; <code>npm run build</code> runs every test before packaging. Live checks via <code>scripts/check-*.js</code> .
Usability	Phone-first layouts; one destination per batch; locked choices when there is only one option; clear refusal messages.
Backup and recovery	SharePoint versioning (major, 500 versions) and the recycle bin (93 days) cover files; flow exports in Git cover flows; config lists can be exported with PnP (Appendix B).
Localisation	English UI. Schedules use Singapore time. Dates are stored in UTC.

11. Design decisions

Table 49. Design decisions

Decision	Reason	Consequence
SharePoint REST, no Microsoft Graph	The client would not grant tenant-wide admin consent.	No Graph features (e.g. Entra group search); everything works with user permissions.
Site collection app catalog	The team has read-only access to the tenant app catalog.	Deploy to <code>/sites/CRS/AppCatalog</code> ; extensions registered by hand.
One site, not several	Multi-site was retired on 26 Jul 2026: same tenant storage quota, no gain.	All libraries on <code>/sites/CRS</code> .
Tagging by proxy (18 Sep 2026)	Uploaders must not hold Edit Items.	Metadata arrives up to a minute after upload; the tagging flow must stay on.
No delete permission for anyone (17 Sep 2026)	Every deletion must be approved and audited.	Deletes are request rows executed by gdc.
Approvers keep Edit Items	SharePoint's Approve Items depends on Edit Items; removing it hid pending files.	An approver-side proxy was built and parked (<code>pendingDecisions.ts</code>).
Content approval with Draft Item Security on the approval libraries	Per-uploader isolation: drafts visible to author and approvers only.	Not possible in Documents (Approve cannot be split from Edit); folders must be approved by a flow.
Separate HC library pair	HC must be invisible to ordinary staff.	Every flow has an HC twin.
Site-collection term store	The client wants terms managed inside the site.	Term GUIDs are site-specific; test sites share the term group or import it.
Terms compared by GUID	Labels are corrupted on this tenant and renamed by the client.	Every term reference stores a GUID.
Additive reconciliation	Folder roots carry automatic Limited Access; a full assertion would strip it.	Revoking means deleting the group.
Approval libraries not searched	Client decision.	Search covers approved and archive libraries only.
Search with a 24-hour REST top-up	The crawler lags behind uploads.	New files appear in search immediately.
Bundled approver email and reminder	Client template; fewer emails for large uploads.	New flows replace NotifyApprovers and the reminder flows on CRS (chapter 12).
Reminders skip site admins (30 Sep 2026)	Client decision.	The first approval email still includes them.
Archive moves after 7 years	Retention requirement; a move keeps the Uniqueld.	Archives are read-only; only C-Level roles have access.
Configuration in lists	Admins change behaviour without a deploy.	Lists must be backed up with the site; code must tolerate missing rows.

12. Known issues and open items

12.1 Ready to roll out on CRS (tested on the test site)

1. Deploy package 1.0.580.0 (CRS runs 1.0.575.0). It is the first build named `sd-guthrie.sppkg`: see the note in Appendix A.4.
2. Bundled approver email: add `ApproverNotifiedAt` (Date and Time, include time) to CRS Submissions; build the flow with `python scripts/build-bundled-approver-flow.py crs`; import it as `gdc`; turn it on; add the switch-over condition `@empty(coalesce(triggerOutputs()?['body/SubmissionFileId'], ''))` to `NotifyApprovers` and `HCNotifyApprovers`.
3. Bundled reminder: `python scripts/build-bundled-reminder-flow.py crs`; import it as `gdc`; turn off the two old reminder flows; turn the new one on.
4. Re-export the changed flows into `PowerAutomateFlowsSDG/` and commit them.

12.2 Waiting on someone

- Client: keep or drop `gdc` as Bcc on the bundled emails (`bcc` in `TARGETS["crs"]`).
- SD Guthrie IT: a mail-flow exception so flow emails can reach addresses outside SD Guthrie.

12.3 Known gaps (accepted for now)

Table 50. Known gaps

Gap	Effect	Fix if needed
Reject does not record who rejected	The rejection email says "Rejected By: not recorded"; Audit Rejected rows name the uploader.	Write <code>ApprovedBy</code> in its own MERGE before the reject, in <code>ApprovalDocument.tsx</code> and <code>BulkApprovePanel.tsx</code> .
Reminders skip site admins who are approvers	Kept as is (30 Sep 2026).	—
Notify request activity fires on every edit	Editing an Approved request row by hand re-sends its emails.	Add a guard column or condition.
Old reminder email link	Points to <code>ApprovalDocument.aspx</code> ; the page is <code>Approval-Document.aspx</code> .	Goes away with the bundled reminder.
Corrupted taxonomy labels	Some stored labels show a number.	Code compares by GUID.
Unused tier columns on all six libraries	Clutter from client testing (4.4).	Remove after checking <code>Levels</code> , <code>PendingLevels</code> and values.
Duplicate and legacy config rows	Two <code>tenantDomains</code> rows; <code>noApprovalEmailFor</code> and <code>bulkUploadRestricted</code> are not read.	Delete the extra rows when convenient.
Site Members group holds Edit	Anyone added bypasses the folder model.	Keep membership to administrators.
Extensions registered by hand	A new site needs the two custom actions added (Appendix A.5).	—
Very large components	<code>FolderManager.tsx</code> (9k lines), <code>Form.tsx</code> (6.7k), <code>MySubmissions.tsx</code> (5.8k).	Change narrowly; extract logic into tested shared modules.

Appendix A. Build, run and deploy

A.1 Software and access

- Node.js 22 (22.14 or later, below 23) through nvm (nvm-windows on Windows); Git; VS Code; Python 3 only to rebuild the generated flows.
- Owner on <https://sdguthrie.sharepoint.com/sites/CRS> (admin pages, deploying).
- The site app catalog [/sites/CRS/AppCatalog](#) (a site Owner can upload and deploy).
- The service account gdc@sdguthrie.com for Power Automate.
- The term store at [/sites/CRS/_layouts/15/termstoremanager.aspx](#).
- A test site set up like CRS (Appendix B).

```
nvm install 22
nvm use 22
npm ci
npx heft trust-dev-cert      (once per machine)
```

A.2 Run locally

1. In `config/serve.json`, set `initialPage` and `serveConfigurations.default.pageUrl` to your test site's workbench (https://<tenant>.sharepoint.com/sites/<test-site>/_layouts/15/workbench.aspx). At handover they point at the live site: never debug against CRS.
2. `npm run start` (Heft dev server on <https://localhost:4321>).
3. Add a web part from the toolbox, or open a real page with the debug query string (9.1).
4. The `hideAppBar` extension has its own serve entry: point its `pageUrl` at a library on the test site.
5. Type check `npx tsc --noEmit`; tests `npx heft test --clean`. Use Heft only (gulp is unused).

A.3 Build

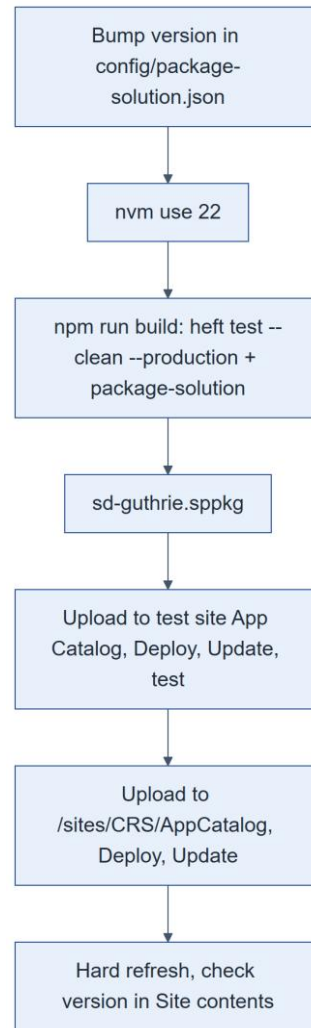


Figure 18. Build and deploy pipeline

1. Add 1 to the last number of version in config/package-solution.json (e.g. 1.0.578.0 → 1.0.579.0). Uploading the same version looks like nothing changed.
2. nvm use 22, then npm run build (runs every test, then packages). Output: sharepoint/solution/sd-guthrie.sppkg.
3. Optional checks (Git Bash): `unzip -p sharepoint/solution/sd-guthrie.sppkg AppManifest.xml | grep -o 'Version="[^\"]*"'` and `unzip -p sharepoint/solution/sd-guthrie.sppkg "ClientSideAssets/<bundle>_*.js" | grep -c "<text you added>"`.

Always use `npm run build`. `heft package-solution` alone packages the JavaScript of the previous build. A new web part needs its id in `componentIds` in `package-solution.json` (a test fails the build if it is missing). Never change the solution id.

A.4 Deploy

1. Deploy to the test site first.
2. Open <https://sdguthrie.sharepoint.com/sites/CRS/AppCatalog>, upload sd-guthrie.sppkg replacing the existing file, click Deploy.
3. Site contents > sd-guthrie-client-side-solution > Update, if offered.
4. Hard refresh (Ctrl+F5) or open a new tab; check the version in Site contents matches the build.

5. A brand-new component can take several minutes to appear with no Update prompt.

First upload after the rename (1.0.580.0): the package used to be called `sd-gatrie.sppkg`. In the App Catalog, rename the existing `sd-gatrie.sppkg` to `sd-guthrie.sppkg` first, then upload, so the upload replaces it. Never delete the old entry: the app's code is served from it. Try it on the test site first.

A.5 Register the extensions

On this tenant the feature did not register the extensions. On a new site, after the first deploy, check `<site>/_api/web/usercustomactions?$select=Title,Location,ClientSideComponentId`. If one is missing, add it once with a POST to `<site>/_api/web/usercustomactions` (two rows load it twice):

Table 51. Custom actions (both present on CRS at capture)

Extension	Title	Location	ClientSideComponentId	Also set
+ New Folder, hides native buttons	CRS New Folder Button	ClientSideExtension.ApplicationCustomizer	c1d2e3f4-a5b6-47c8-d9e0-f1a2b3c4d5e6	—
Approve or reject panel	CRS Bulk Approve	ClientSideExtension.ListViewCommandSet.CommandBar	6d1a9f47-3c82-4e15-b7a0-9f2e5c8d4136	RegistrationId "101", RegistrationType 1

Appendix B. Set up a test site

Copy the structure of CRS (lists, libraries, columns, views, pages, config rows) with PnP PowerShell, then deploy the app, create the groups and import the flows. No documents are copied. Plan about a day; these steps have not yet been run end to end on a fresh site, so check each one.

1. Create a Team site (e.g. /sites/CRS-Test); add the service account to its Owners; ask a SharePoint admin for a site app catalog: `Add-SPOSiteCollectionAppCatalog -Site <test site URL>`.
2. Terms. Same tenant: in CRS's term store, open the site collection term group and add the test site under Site Collection Access, so it uses the same term sets and GUIDs. Another tenant: `Export-PnPTermGroupToXml / Import-PnPTermGroupFromXml` (keeps term ids).
3. Deploy the app (Appendix A.4), add it to the site, register the extensions if missing (A.5).
4. Copy the structure (needs an Entra app registration once per tenant, `Register-PnPEntraIDAppForInteractiveLogin`). Leave Group Map and Folder Map empty (the app refills them); do not copy site security.
5. Check the settings: content approval on for both approval libraries (drafts: approvers + author) and off for both approved libraries; BulkImport visible; NextReminderAt on both approval libraries; ApproverNotifiedAt on CRS Submissions; every page published and named as on CRS.
6. In the app, as Owner: Group Management › Bulk Provisioning; add test users (an uploader and an approver who are not site admins); Folder Administration › Folder Reconciliation.
7. Import the flows pointed at the test site (D.5). Minimum set: Apply pending tags, Approve new folders (+ HC), NotifyApprovers, Auto-route (+ HC), Auto-approve bulk imports (+ HC).
8. Smoke test: upload as the uploader (metadata fills in, approver gets an email), approve (the file moves within about 5 minutes), check My Submissions.

```
Connect-PnPOnline -Url https://sdguthrie.sharepoint.com/sites/CRS -Interactive -ClientId <app id>
Get-PnPSiteTemplate -Out crs.pnp -Handlers Fields,ContentTypes,Lists,Pages,Navigation -
IncludeAllClientSidePages
Add-PnPDataRowsToSiteTemplate -Path crs.pnp -List "CRS Config"
Add-PnPDataRowsToSiteTemplate -Path crs.pnp -List "CRS Term Abbreviation"
Connect-PnPOnline -Url <test site URL> -Interactive -ClientId <app id>
Invoke-PnPSiteTemplate -Path crs.pnp
```

Appendix C. Troubleshooting

Most problems turned out to be a stale browser tab, the wrong site, or a flow. Check in this order:

1. Which site? Live and test sites have different list GUIDs, term GUIDs and data.
2. Which build? Hard refresh; compare the app version in Site contents with package-solution.json.
3. The real error: browser F12 Console, and the Network tab's status and response body.
4. Is it a flow? Open its run history as gdc; check the gdc connections (D.2).
5. Live data: run a `scripts/check-*` or `diagnose-*` script in the F12 console.

Table 52. Symptoms and causes

Symptom	Usual cause
Web part blank, no error	A React hook placed below an early return (React error #310).
A library "disappeared" after a rename	New title missing from <code>src/shared/naming.ts</code> (and <code>CRS_LIBRARIES</code>).
Uploaded file has no metadata	Apply pending tags failed or is off; check the record's <code>TagStatus</code> / <code>TagError</code> .
No approval or reminder email	Flow off or failed; gdc connection broken; the person is the uploader or (reminders) a site admin; or an outside address blocked by the tenant.
Uploaders cannot see each other's folders	Folder still pending: Approve new folders must be on.
New member has no access	They must sign out and in; or reconciliation has not run for their group.
Items or groups missing from a screen	A \$top-capped read; filter server-side or page it.
Write returned 200 but nothing changed	<code>validateUpdateListItem</code> reported a field error inside the 200 (HasException).
Search results stale	Re-index the library; check the reindex flow.
Everything automated stopped at once	gdc removed from Owners, or its connections need re-authentication.

Appendix D. Flow operations

D.1 Rules for editing flows

- Build, import and edit flows signed in as gdc, and export to PowerAutomateFlowsSDG/ after any change.
- HTTP header names have no colon (Accept, not Accept:); with a colon the header silently does nothing.
- Always send Accept: application/json;odata=nometadata, or ?['value'] comes back empty, except the two deliberate owner/admin lookups in NotifyApprovers (adding the header would switch that exclusion on).
- Writing a field re-triggers every other created-or-modified flow on that list (8.3).
- The connector's Update item rewrites every column; for one field use an HTTP MERGE.
- Never MERGE OData__ModerationStatus together with another field.
- Type expressions in the fx box and never press Enter in it; a 0 typed into a Condition is text.
- A green run does not prove the logic is right; re-export to confirm an edit landed.
- gdc must stay in the site's Owners group.

D.2 If gdc's password changes

1. Sign in to <https://make.powerautomate.com> as gdc@sdguthrie.com.
2. Open Connections. Any showing Reconnect or Fix connection is broken; the two that matter are GDC Proxy - Outlook and GDC Proxy - SharePoint.
3. Reconnect and sign in as gdc with the new password, never as yourself (the flows would then act and send email as you).
4. Open one email flow's latest run to confirm it is green. Resubmit failed runs from the run history if the missed actions matter.

D.3 Export and back up

1. Open the flow as gdc › Export › Package (.zip).
2. Save it into PowerAutomateFlowsSDG/ replacing the old one, and commit it.

D.4 Import or update

1. My flows › Import › Import Package (Legacy) › choose the zip.
2. For each resource pick Create as new (first time) or Update.
3. Under Related resources pick the gdc connections.
4. Import, open the flow, save, turn it on.

D.5 Point a flow at another site

Each export names CRS's site URL and list GUIDs (8.6). Change every SharePoint action's Site Address and List Name in the designer, or replace the URL and each GUID in `definition.json` before importing. Get the other site's GUIDs from `<site>/_api/web/lists?$select=Title,Id&$filter=Hidden eq false`.

D.6 Make the HC copy of a flow

1. Open the normal flow › Save As, name it with HC.

2. Change every library reference to its HC twin: triggers, Get items, HTTP URIs (`getbytitle(...)` and `lists(guid'...')`), and links in emails. Role APR becomes APRHC.
3. Re-export and search `definition.json` for the normal library names and GUIDs: every HC copy so far has kept at least one.

D.7 Rebuild a generated flow

1. Edit TARGETS in `scripts/build-bundled-approver-flow.py` (site URL, list GUIDs, bcc, base export path and output path). The reminder script reuses it.
2. Run `python scripts/build-bundled-approver-flow.py crs` or `python scripts/build-bundled-reminder-flow.py crs`.
3. Import the zip as gdc (Create as new the first time, Update afterwards), pick the connections, save, turn on.

D.8 Turn a flow off safely

- A recurrence flow that is off catches up on its next run. A trigger flow that is off does not see changes made while it was off.
- Never turn off Apply pending tags, Auto-route or Approve new folders on the live site except for a short, announced window.